



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

Proyecto AnzenCore

Curso: Calidad y Pruebas de Software

Docente: Patrick Jose Cuadros Quiroga

Integrantes:

Arocutipa Arocutipa, Gian Franco (2023076790)

Perez Peralta, Fabrizio Salvador Elias (2023077476)

Tacna – Perú

2026

CONTROL DE VERSIONES					
Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	G.A. / F.P.			27/04/2026	Versión Original

AnzenCore
Documento de Especificación de Requerimientos de
Software

Versión {1.0}

CONTROL DE VERSIONES					
Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	G.A. / F.P.			27/04/2026	Versión Original

ÍNDICE GENERAL

INTRODUCCIÓN	5
I. Generalidades de la empresa	6
A. Nombre de la empresa.....	6
B. Visión.....	6
C. Misión	6
D. Organigrama	6
II. Visionamiento de la empresa	6
A. Descripción de la empresa	6
B. Objetivos de negocios	7
C. Objetivos de diseño	7
D. Alcance del proyecto.....	8
E. Viabilidad del sistema	8
F. Información obtenida del Levantamiento de Información	9
III. Análisis de Procesos	9
A. Diagrama del Proceso Actual - Diagrama de actividades	9
B. Diagrama del Proceso Propuesto - Diagrama de actividades Inicial	11
IV. Especificación de Requerimientos de Software.....	13
A. Cuadro de Requerimientos funcionales Inicial	13
B. Cuadro de Requerimientos No funcionales	14
C. Cuadro de Requerimientos funcionales Final	15
D. Reglas de Negocio	15
V. Fase de Desarrollo	16
A. Perfiles de Usuario	16
B. Modelo Conceptual	17
a. Diagrama de Paquetes	17
b. Diagrama de Casos de Uso	18
c. Escenarios Casos de Uso	19
C. Modelo Lógico.....	21
a. Análisis de Objetos	21
b. Diagrama de Actividades con objetos.....	21
c. Diagrama de Secuencia.....	38
d. Diagrama de Clases	46
CONCLUSIONES	48

RECOMENDACIONES	49
BIBLIOGRAFÍA.....	49

INTRODUCCIÓN

El presente documento describe la Especificación de Requerimientos de Software (ERS) para el sistema AnzenCore, una plataforma web con componente móvil (APK) orientada a la auditoría de seguridad de aplicaciones Android mediante un motor de ingeniería inversa sobre bytecode Dalvik/DEX. AnzenCore desempaqueta archivos APK, extrae y desofusca el bytecode DEX, y detecta vulnerabilidades OWASP Mobile Top 10 sin ejecutar la aplicación analizada (análisis estático). Se implementa con arquitectura MVC en Python (FastAPI + Streamlit) y base de datos Supabase (PostgreSQL 15).

AnzenCore propone un modelo de análisis centralizado: el usuario sube su APK a la plataforma web y el servidor aplica ingeniería inversa sobre el binario: desempaqueta el APK, extrae el pool de strings del bytecode DEX mediante parseo binario (struct), aplica algoritmos de desofuscación sobre strings y clases ofuscadas con ProGuard/R8, y ejecuta detectores de vulnerabilidades sobre el código resultante. Los hallazgos se presentan en un dashboard interactivo con score de seguridad, severidad OWASP y recomendaciones de mitigación.

El sistema se implementa con arquitectura MVC en Python, utilizando FastAPI para el backend API y Streamlit para el dashboard web, con Supabase (PostgreSQL) como base de datos.

I. Generalidades de la empresa

A. Nombre de la empresa

AnzenCore — Plataforma de Auditoría de Seguridad Móvil

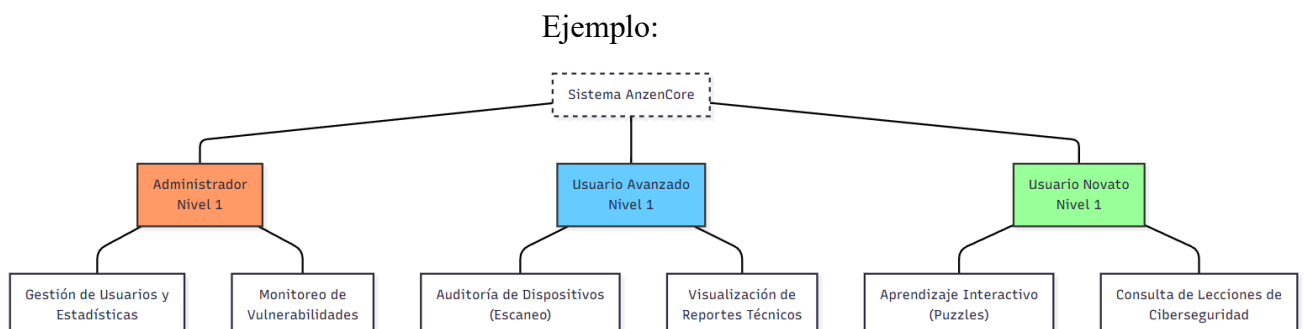
B. Visión

Ser la plataforma de referencia en auditoría de seguridad de aplicaciones Android para empresas de desarrollo de software en Latinoamérica, ofreciendo integración empresarial mediante API REST, reportes ejecutivos PDF y análisis continuo de APKs mediante ingeniería inversa sobre bytecode Dalvik/DEX.

C. Misión

Proveer a las empresas de desarrollo Android una plataforma SaaS de análisis de seguridad de nivel profesional que aplica ingeniería inversa sobre bytecode DEX, detecta vulnerabilidades OWASP Mobile Top 10 incluso en código ofuscado con ProGuard/R8, e integra los resultados en los pipelines CI/CD del cliente mediante API REST documentada (OpenAPI/Swagger).

D. Organigrama



II. Visionamiento de la empresa

A. Descripción de la empresa

Las empresas de desarrollo Android enfrentan el reto de auditar la seguridad de sus APKs antes de publicarlas en Google Play. Las herramientas existentes (MobSF, Checkmarx, AppScan) requieren instalación local, conocimiento avanzado de ingeniería inversa o licencias costosas (\$60,000+/año). AnzenCore resuelve esto ofreciendo análisis automatizado vía API REST: la empresa envía la APK, el motor IR desofusca el bytecode DEX y devuelve un reporte JSON/PDF con hallazgos OWASP, evidencias y recomendaciones de mitigación, sin necesidad de infraestructura propia ni instalación.

B. Objetivos de negocios

N°	Objetivo	Indicador de Éxito
1	Ofrecer auditoría de seguridad Android sin instalación permanente	APK temporal funcional con desinstalación automática
2	Integrar AnzenCore en el pipeline CI/CD de empresas de desarrollo Android mediante API REST documentada	Al menos 3 empresas integradas vía API REST durante el primer año de operación
3	Generar historial de seguridad trazable por usuario	Dashboard web con gráfico de evolución de puntuación
4	Establecer contratos de servicio (plan Empresarial S/.89.90/\$24.00 USD/mes) con empresas del sector tecnológico latinoamericano	Mínimo 2 contratos Empresarial activos al finalizar el primer semestre de operación

C. Objetivos de diseño

N°	Objetivo de Diseño
1	Arquitectura MVC en Python (FastAPI + Streamlit) con separación clara de capas (Model, View, Controller)
2	Análisis de APK procesado en el servidor mediante motor Python de ingeniería inversa Dalvik

3	Dashboard web interactivo accesible desde cualquier navegador sin instalación (Streamlit)
4	Motor IR: extracción del bytecode DEX, desofuscación de strings/clases ofuscadas con ProGuard/R8, ejecución de detectores de vulnerabilidades OWASP
5	Agente Android (Kivy, API 26+): escaneo local de permisos peligrosos, certificados de red y configuración de seguridad del dispositivo

D. Alcance del proyecto

El sistema AnzenCore comprende los siguientes módulos:

Módulo	Descripción	Tecnología
Plataforma Web (MVC)	Registro, dashboard, historial de análisis, comunidad de usuarios online	Python (Streamlit)
API Backend	Endpoints REST para análisis de APK, código, URLs y repositorios	Python (FastAPI)
Análisis de APK	Análisis server-side de APKs: permisos, secretos, vulnerabilidades	Python (motor Dalvik)
Motor de Ingeniería Inversa	Desempaquetado del APK, parseo del bytecode DEX, desofuscación de strings y clases ofuscadas, ejecución de 8+ detectores de vulnerabilidades OWASP Mobile Top 10 sobre el código desofuscado. Resultado: lista de hallazgos con evidencia, severidad, CWE y categoría OWASP.	Python stdlib (zipfile, struct, re)
Agente Móvil Android	Aplicación Android (Kivy, API 26+) que escanea el dispositivo local: permisos peligrosos de apps instaladas, certificados de red, configuración del sistema. Envía reporte JSON al backend para persistencia y visualización.	Python (Kivy, API 26+)
Base de Datos	Almacenamiento de usuarios, escaneos, hallazgos y artefactos	Supabase (PostgreSQL)

E. Viabilidad del sistema

El sistema propuesto es viable técnica, económica y operativamente:

- **Técnicamente**, el equipo cuenta con conocimientos de Python, FastAPI, Streamlit y arquitectura MVC. Las tecnologías son de código abierto y no generan costos de licencia. Supabase ofrece tier gratuito para desarrollo académico.
- **Económicamente**, AnzenCore opera con un único plan Empresarial: S/.89.90/mes (equivalente a \$24.00 USD/mes), orientado a empresas de desarrollo Android que requieren auditorías de seguridad continuas e integración API REST. El despliegue se realiza en Azure Container Apps mediante Terraform. B/C = 6.38, VAN = S/.46,121, TIR $\approx$ 158% (horizonte 3 años, COK 10%). Análisis completo en FD01.
- **Operativamente**, el sistema es accesible desde cualquier navegador web (Streamlit). La APK de análisis se distribuye desde la plataforma web como descarga directa, y el análisis se procesa en el servidor.

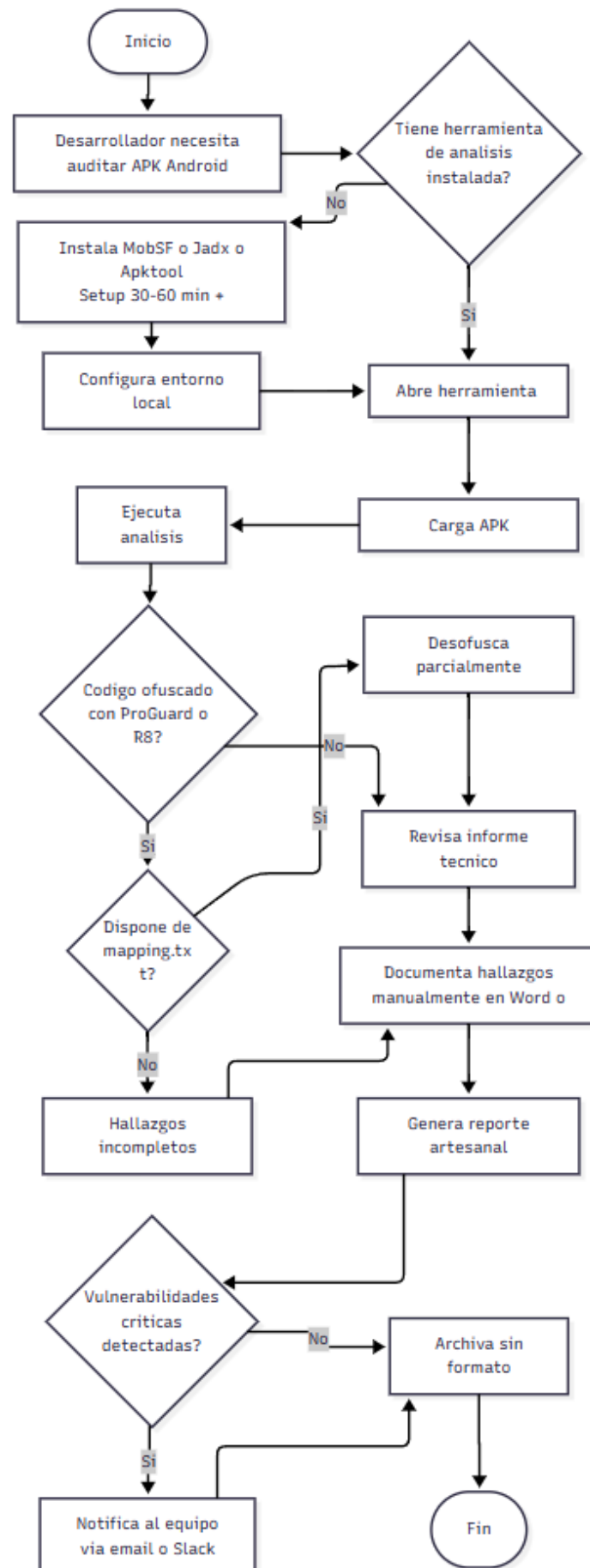
F. Información obtenida del Levantamiento de Información

Técnica	Fuente	Hallazgo Principal
Revisión de literatura	OWASP Mobile Top 10	Permisos excesivos y almacenamiento inseguro son las amenazas más comunes en Android
Análisis de competidores	Apps Play Store (antivirus)	Requieren instalación permanente y no ofrecen integración empresarial vía API REST

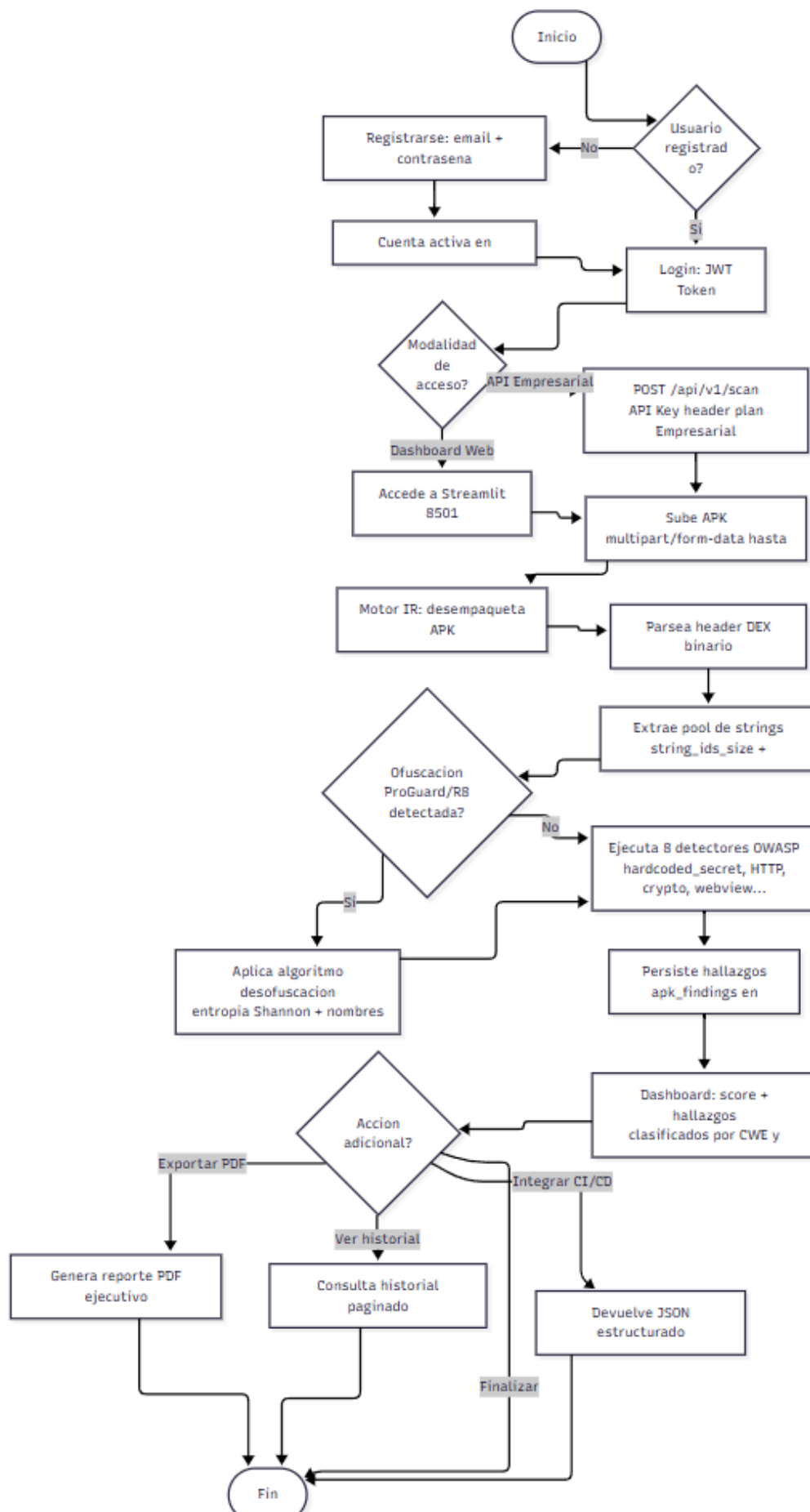
Encuesta informal	Estudiantes universitarios	85% desconoce qué permisos tienen sus apps instaladas
Análisis de sistema	Android Developer Docs	API de Android permite acceso a metadatos de seguridad sin datos personales

III. Análisis de Procesos

A. Diagrama del Proceso Actual - Diagrama de actividades



B. Diagrama del Proceso Propuesto - Diagrama de actividades Inicial



IV. Especificación de Requerimientos de Software

A. Cuadro de Requerimientos funcionales Inicial

ID	Módulo	Requerimiento Funcional	Prior.
RF-01	Autenticación	El sistema debe permitir registro de usuarios con nombre de usuario y contraseña	Alta
RF-02	Autenticación	El sistema debe permitir inicio y cierre de sesión seguro	Alta
RF-03	APK	El sistema debe permitir la descarga de la APK de análisis desde la plataforma web	Alta
RF-04	Análisis APK	El servidor debe analizar la APK subida: permisos peligrosos, secretos hardcodeados, código ofuscado, vulnerabilidades y metadatos del paquete	Alta
RF-05	Análisis APK	El sistema debe subir la APK por HTTPS y procesarla en el servidor sin almacenar datos personales del usuario	Alta
RF-06	Dashboard	El sistema debe mostrar puntuación de seguridad (severity_max) y hallazgos del APK	Alta
RF-07	Dashboard	El sistema debe listar vulnerabilidades/hallazgos clasificados por criticidad	Alta
RF-08	Motor IR	El sistema aplica algoritmos de desofuscación sobre los strings y clases extraídos del bytecode DEX y ejecuta los detectores de vulnerabilidades sobre el código desofuscado, generando hallazgos con evidencia, CWE y OWASP.	Alta
RF-09	Dashboard	El sistema permite exportar los hallazgos del análisis IR como reporte PDF con evidencias extraídas, severidad, CWE, categoría OWASP y recomendaciones de mitigación. Disponible en planes Académico, Profesional y Empresarial.	Media
RF-10	Historial	El sistema almacena el historial de escaneos APK del usuario con paginación y permite comparar dos versiones de una	Alta

		misma APK (por package name) para medir la evolución de la seguridad entre releases.	
RF-11	Integración Empresarial	El sistema provee una API REST documentada (OpenAPI/Swagger) para que empresas integren AnzenCore en sus pipelines CI/CD: endpoint POST /api/v1/scan acepta APK como multipart/form-data y devuelve el reporte JSON con hallazgos, severidad OWASP, CWE y recomendaciones de mitigación.	Alta
RF-12	Historial	El sistema debe almacenar y mostrar el historial de escaneos APK por usuario	Alta
RF-13	Comunidad	El sistema debe mostrar usuarios conectados en tiempo real	Media

B. Cuadro de Requerimientos No funcionales

ID	Categoría	Requerimiento No Funcional
RNF-01	Seguridad	Todos los reportes de análisis deben transmitirse cifrados con TLS 1.2+
RNF-02	Privacidad	La APK no debe leer, acceder ni almacenar contactos, mensajes o fotos
RNF-03	Rendimiento	El análisis del dispositivo debe completarse en menos de 60 segundos
RNF-04	Disponibilidad	La plataforma web debe estar disponible el 99% del tiempo (24/7)
RNF-05	Usabilidad	El dashboard debe ser responsivo y usable en pantallas desde 320px
RNF-06	Compatibilidad	La APK debe ser compatible con Android 8.0 (API 26) o superior
RNF-07	Mantenibilidad	El backend debe seguir estrictamente el patrón MVC con Python (FastAPI + Streamlit)
RNF-08	Escalabilidad	La arquitectura debe soportar hasta 10,000 usuarios concurrentes
RNF-09	Portabilidad	La plataforma web debe funcionar en Chrome, Firefox, Safari y Edge

C. Cuadro de Requerimientos funcionales Final

ID	CU	Actor	Descripción Refinada	Prior.
RF-01	CU-01	Usuario	Registrarse con nombre de usuario y contraseña. Validar usuario único.	Alta
RF-02	CU-02	Usuario	Iniciar sesión con credenciales. Generar token de sesión.	Alta
RF-03	CU-03	Usuario	Descargar APK de análisis precompilada desde la plataforma web.	Alta
RF-04	CU-04	Sistema	Analizar APK subida: permisos peligrosos, secretos hardcodeados, URLs inseguras, criptografía débil, código ofuscado y metadatos del paquete (server-side Python).	Alta
RF-05	CU-05	Sistema	Recibir APK por HTTPS, procesarla en servidor y persistir hallazgos en Supabase (apk_scans, apk_findings, apk_artifacts).	Alta
RF-06	CU-06	Sistema	Calcular severidad máxima y conteo de hallazgos; mostrar en dashboard.	Alta
RF-07	CU-07	Usuario	Ver dashboard con escaneos APK: score, hallazgos clasificados por severidad y artefactos extraídos. Exportar reporte en PDF.	Alta
RF-08	CU-08	Usuario	El motor IR extrae strings del bytecode DEX, aplica desofuscación (strings ProGuard/R8) y ejecuta 8+ detectores de vulnerabilidades. Hallazgos persistidos en apk_findings con evidencia, CWE y categoría OWASP.	Alta
RF-09	CU-09	Usuario	Exportar reporte PDF con hallazgos del análisis IR, evidencias extraídas por ingeniería inversa, severidad y recomendaciones de mitigación. Disponible en planes Académico, Profesional y Empresarial.	Media
RF-10	CU-10	Usuario	Ver historial paginado de escaneos APK propios. Comparar dos versiones de una misma aplicación (por package name) para medir evolución de la seguridad entre releases.	Alta
RF-11	CU-11	Usuario	La empresa cliente envía la APK al endpoint POST /api/v1/scan con autenticación API Key (plan Empresarial). El motor IR procesa el binario y devuelve JSON con hallazgos OWASP, evidencias DEX y recomendaciones. Permite integración directa en pipelines Jenkins, GitHub Actions o GitLab CI.	Alta
RF-12	CU-12	Usuario	Ver historial de escaneos APK propios con fecha, tamaño y estado.	Alta

RF-13	CU-13	Sistema	Mostrar usuarios conectados en tiempo real (ping cada 30 segundos a Supabase).	Media
-------	-------	---------	--	-------

D. Reglas de Negocio

ID	Regla de Negocio	Descripción
RN-01	APK Análisis Server-Side	La APK es analizada en el servidor por el motor Python. El usuario solo la sube; no instala nada en su dispositivo.
RN-02	Privacidad	El sistema no almacena el contenido de la APK; solo guarda los hallazgos y metadatos del análisis en Supabase.
RN-03	Severidad de Hallazgos	Los hallazgos se clasifican por severidad (critical, high, medium, low, info). La severidad máxima determina el estado del escaneo.
RN-04	Unicidad de Escaneo	Cada APK subida genera un registro único en apk_scans con hash SHA-256 del archivo, usuario y timestamp.
RN-05	Historial Inmutable	Los escaneos históricos se conservan. No se eliminan aunque el usuario realice nuevos análisis.
RN-06	Desofuscación IR	El motor IR aplica análisis de nombres cortos (ProGuard/R8: clases a, b, c...) y análisis de entropía de strings para detectar ofuscación. Los strings del pool DEX son analizados directamente: no se alteran por la ofuscación de identificadores, lo que permite detectar vulnerabilidades en cualquier APK.
RN-07	Privacidad APK	El motor IR procesa el APK en memoria (BytesIO). El binario APK se descarta tras el análisis: no se almacena en la base de datos. Solo se persisten hallazgos, artefactos, permisos y metadatos extraídos por ingeniería inversa.

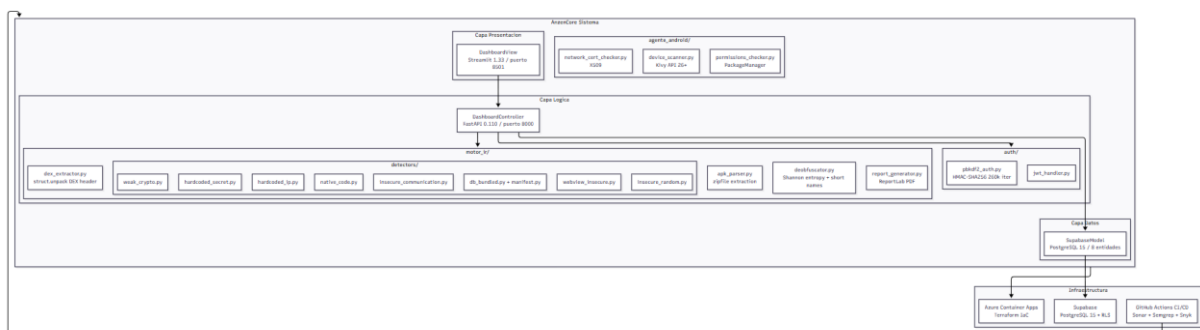
V. Fase de Desarrollo

A. Perfiles de Usuario

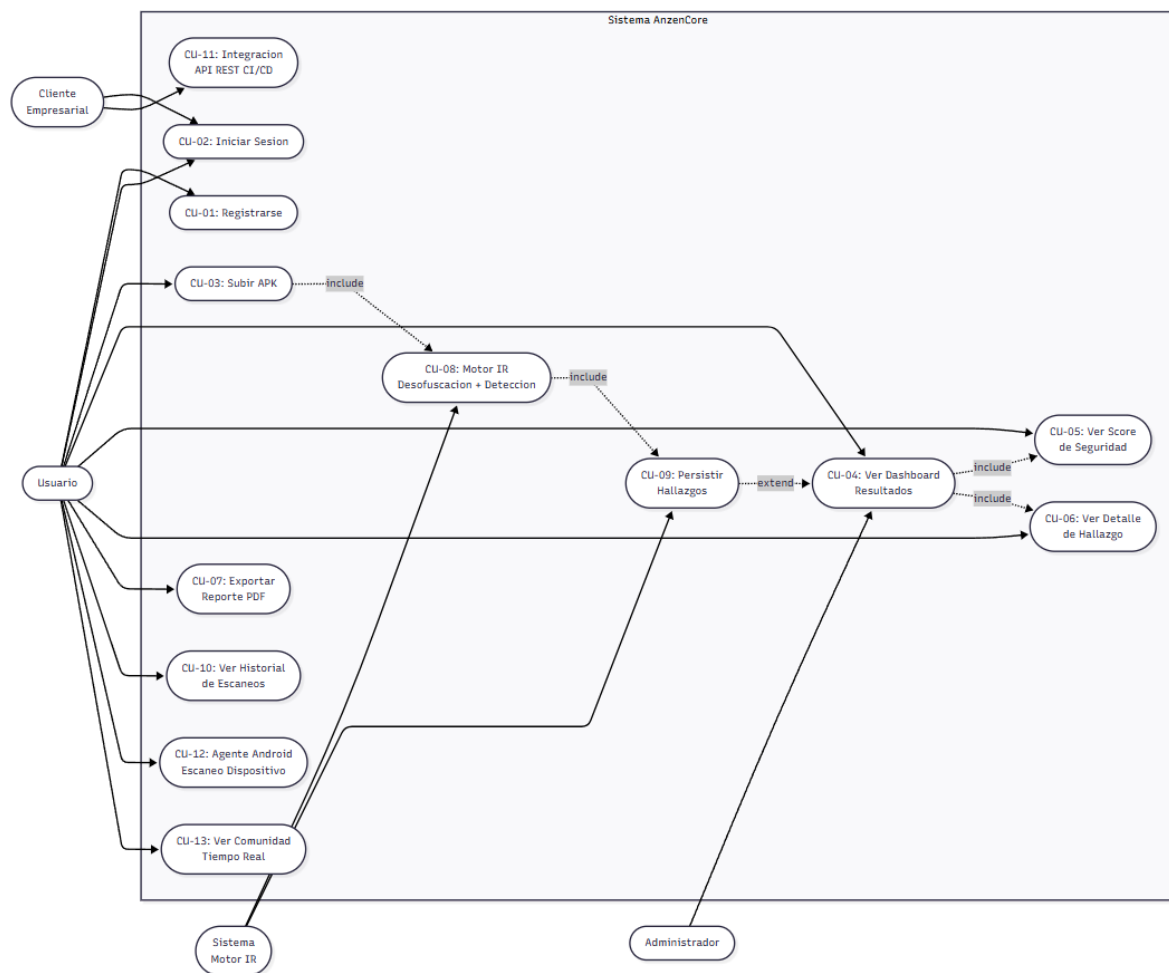
Perfil	Descripción	Acceso
Usuario Final (Novato)	Usuario sin conocimientos técnicos avanzados que desea analizar una APK para detectar vulnerabilidades de seguridad. Usa el dashboard, el motor IR y exportación PDF.	Dashboard APK, Motor IR, comunidad
Usuario Final (Avanzado)	Usuario con conocimientos de seguridad que usa AnzenCore para auditoría de APKs, análisis estático de código y seguimiento de historial de hallazgos.	Dashboard APK, historial, Motor IR, Exportación PDF
Administrador	Administra la base de datos Supabase. No tiene panel web dedicado en la implementación actual.	Supabase dashboard (externo al sistema)

B. Modelo Conceptual

a. Diagrama de Paquetes



b. Diagrama de Casos de Uso



c. Escenarios Casos de Uso

CU-01: Registrarse

Campo	Descripción
ID	CU-01
Nombre	Registrarse
Actor Principal	Usuario
Precondición	El usuario no tiene cuenta registrada
Flujo Principal	1. Usuario accede a /register en el dashboard web. 2. Ingresar email y contraseña. 3. Sistema verifica que el email no exista en la tabla usuarios. 4. Sistema aplica PBKDF2-HMAC-SHA256 (260 000 iteraciones) con sal aleatoria. 5. Sistema persiste el usuario con hash+sal en Supabase. 6. Redirige al dashboard principal con sesión iniciada.
Flujo Alternativo	3a. Email ya existe: sistema muestra error y solicita otro email

Postcondición	Usuario registrado con PBKDF2 y sesión JWT iniciada
Prioridad	Alta

CU-03: Subir APK para Análisis

Campo	Descripción
ID	CU-03
Nombre	Subir APK para Análisis
Actor Principal	Usuario
Precondición	Usuario autenticado en la plataforma web
Flujo Principal	1. Usuario accede al formulario de subida. 2. Selecciona archivo APK (hasta 50 MB). 3. Sistema verifica magic bytes ZIP (PK signature) y extensión .apk. 4. Sistema calcula hash SHA-256 del archivo. 5. Sistema crea registro en apk_scans con status=processing. 6. Motor IR es invocado en segundo plano (BackgroundTask). 7. Sistema retorna scan_id para polling de resultados.
Flujo Alternativo	3a. El archivo no es un APK válido: sistema rechaza la carga y muestra un error
Postcondición	APK recibida y registrada para su análisis
Prioridad	Alta

CU-04/05: Ver Dashboard / Resultados del Análisis

Campo	Descripción
ID	CU-04
Nombre	Ver Dashboard / Resultados del Análisis
Actor Principal	Usuario
Precondición	Usuario autenticado con al menos un APK analizado

Flujo Principal	1. Usuario accede a /dashboard/scan_id. 2. DashboardController carga hallazgos desde apk_findings (Supabase). 3. Calcula security_score: 100 – penalizaciones por severidad (CRÍTICO –20, ALTO –10, MEDIO –5, BAJO –1). 4. Renderiza score con color (rojo <40, naranja 41-70, verde >70). 5. Muestra tabla de hallazgos filtrable por categoría OWASP y severidad. 6. Muestra gráfico de barras por categoría OWASP.
Flujo Alternativo	2a. Sin análisis previo: muestra botón "Subir mi primera APK"
Postcondición	Usuario visualiza score, hallazgos clasificados OWASP y CWE
Prioridad	Alta

CU-07: Exportar Reporte PDF

Campo	Descripción
ID	CU-07
Nombre	Exportar Reporte PDF
Actor Principal	Usuario
Precondición	Usuario autenticado con plan Empresarial y al menos un análisis
Flujo Principal	1. Usuario hace clic en "Exportar PDF" en el dashboard. 2. DashboardController verifica que el plan del usuario sea Empresarial. 3. Carga hallazgos y metadatos del scan desde Supabase. 4. ReportGenerator.generate_pdf() construye el PDF con ReportLab: portada, score ejecutivo, tabla de hallazgos con evidencia, recomendaciones OWASP. 5. Persiste registro en report_exports. 6. Retorna PDF para descarga directa via st.download_button.
Flujo Alternativo	2a. Plan no Empresarial: muestra mensaje de upgrade requerido
Postcondición	PDF ejecutivo descargado con hallazgos, evidencias y recomendaciones
Prioridad	Alta

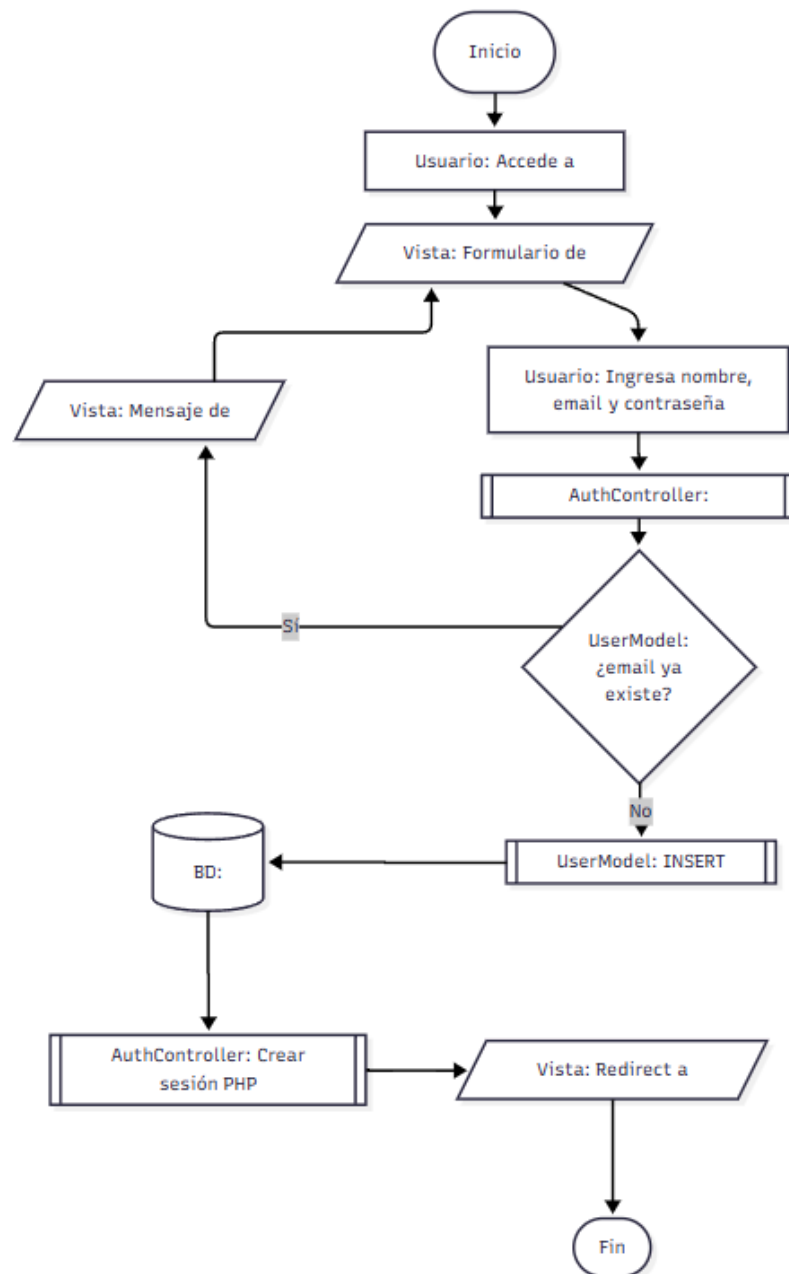
C. Modelo Lógico

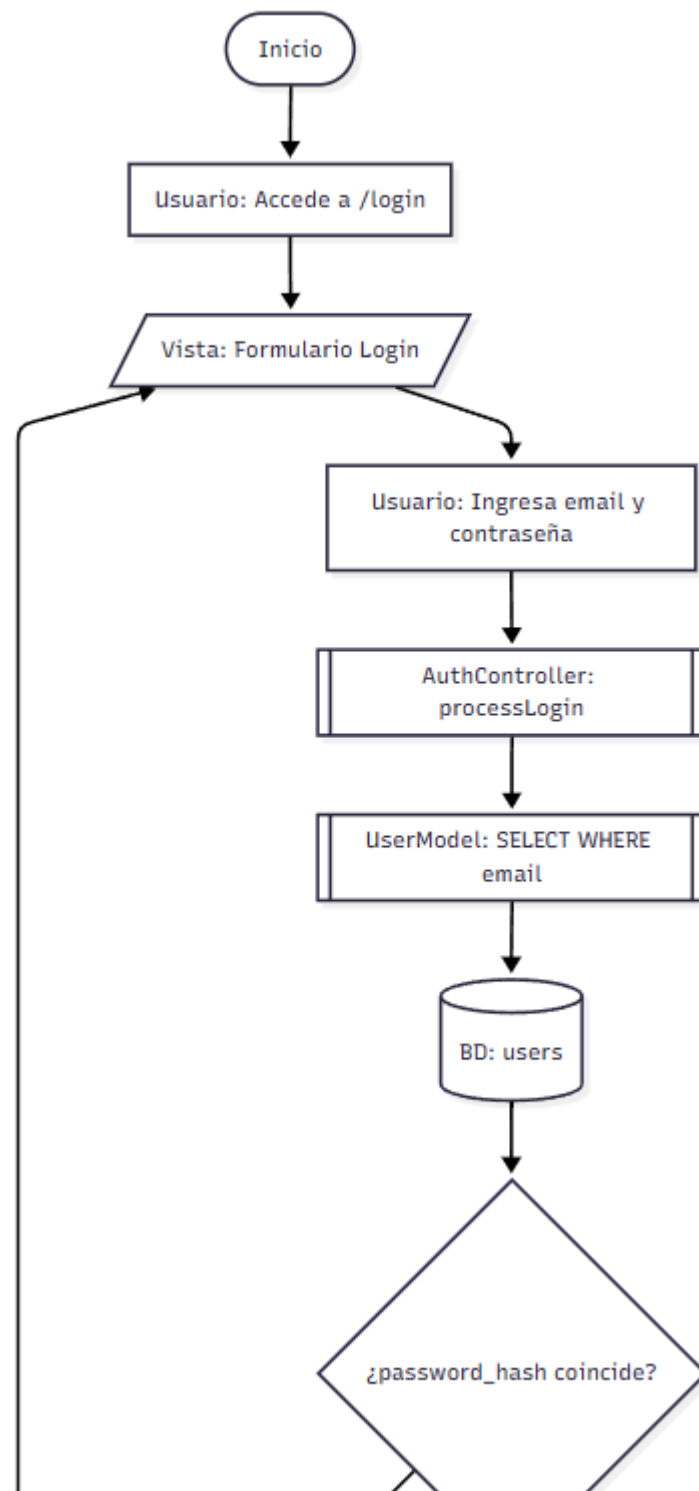
a. Análisis de Objetos

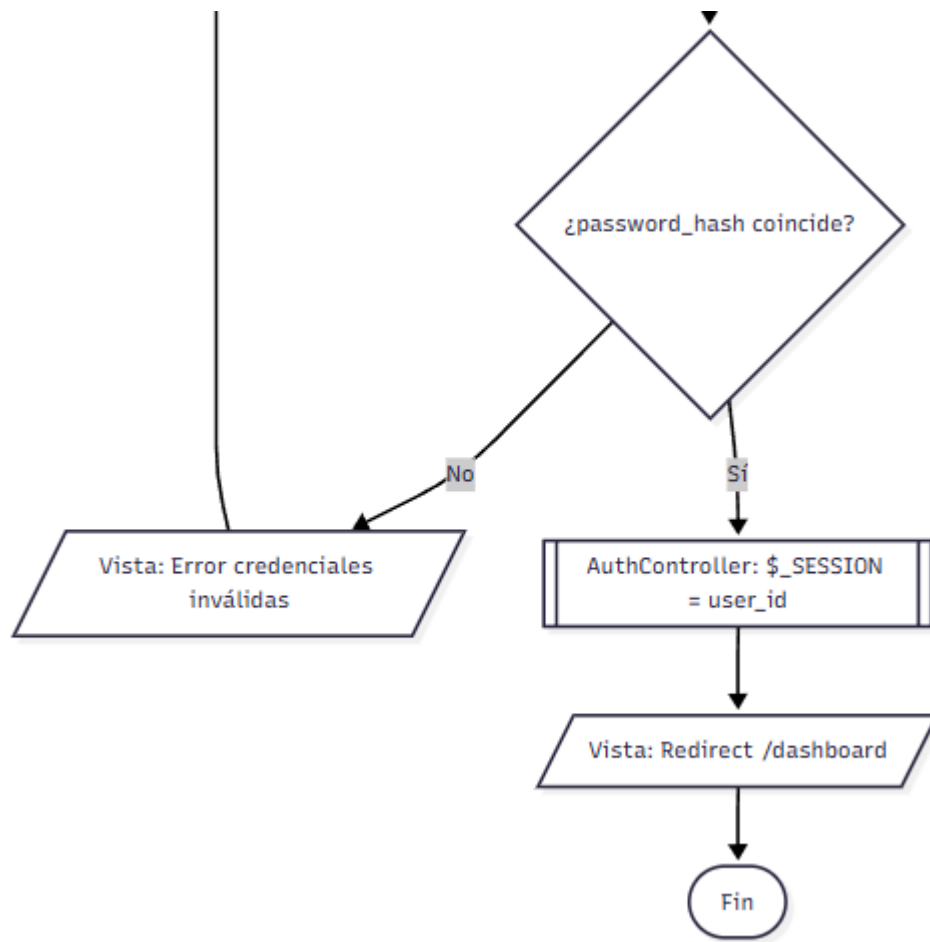
Clase	Atributos Principales (Supabase)	Métodos / Responsabilidad
Usuario	id, email, password_hash, salt, plan, api_key, last_seen, created_at, is_active	authenticate(), register(), update_ping(), get_online_users()
ApkScan	id, user_id, filename, package_name, version_name, version_code, security_score, status, file_size_bytes, findings_count, created_at	create_apk_scan(), update_apk_scan(), get_apk_scans()
ApkFinding	id, scan_id, detector, severity, title, evidence, cwe, owasp_category, line_number, recommendation, type	create_apk_findings(), get_apk_findings()
ApkArtifact	id, scan_id, artifact_type, content, created_at	create_apk_artifacts(), get_apk_artifacts()
Vulnerabilidad	id, code, name, description, owasp_category, cwe, severity, mitigation, references	save_vulnerabilities(), get_vulnerabilities()
ReportExport	id, scan_id, user_id, format, download_url, created_at	create_report_export(), build_pdf()

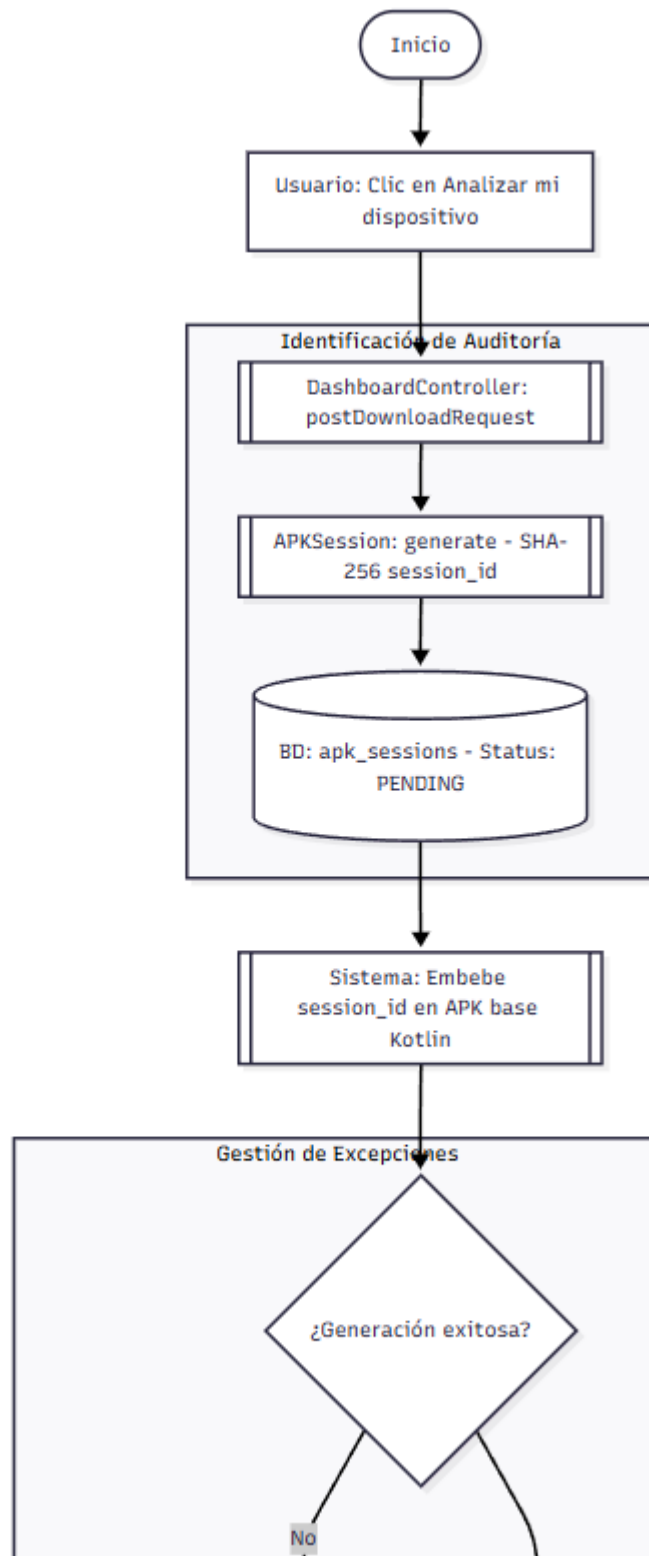
b. Diagrama de Actividades con objetos

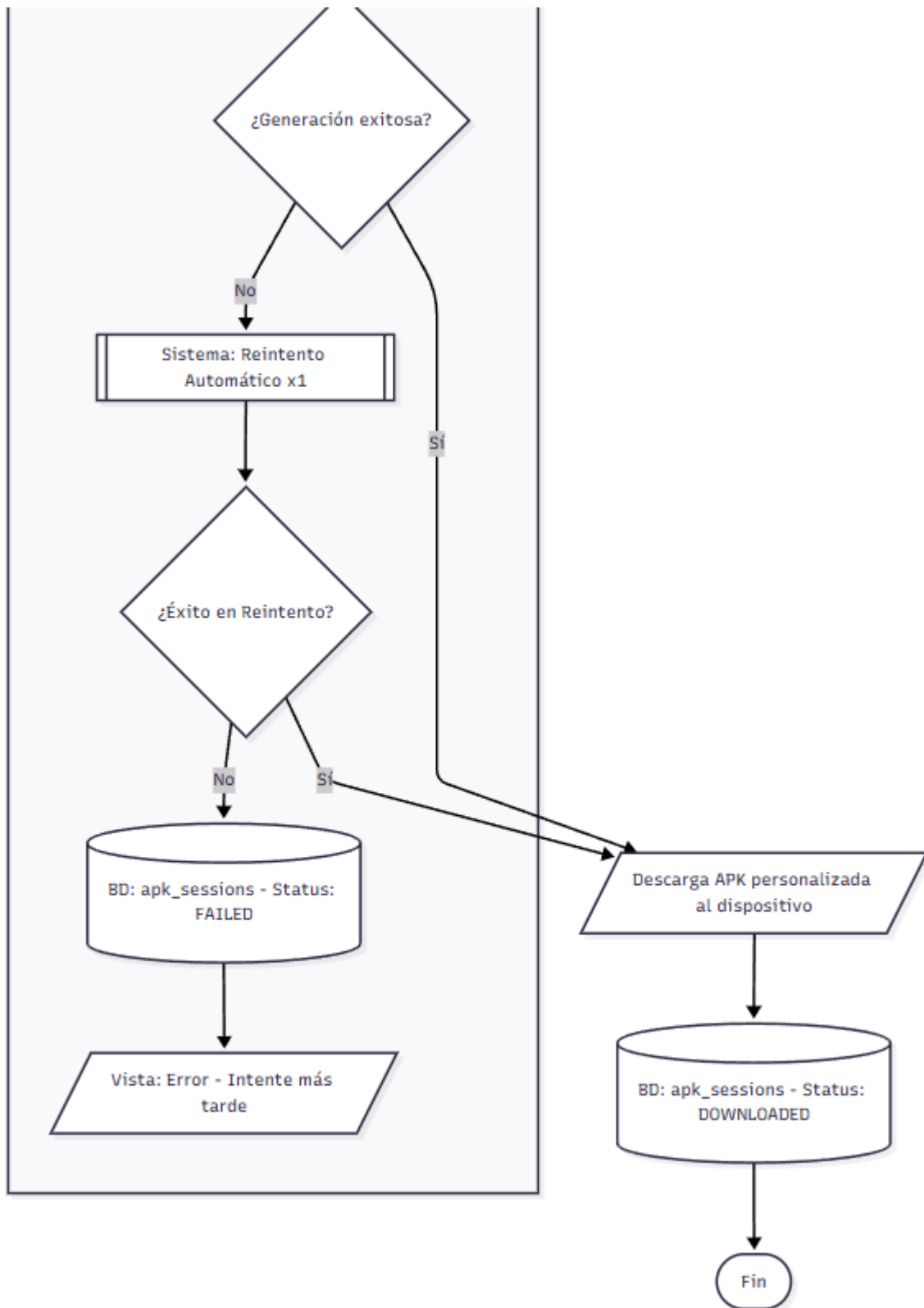
CU-01: Registrarse

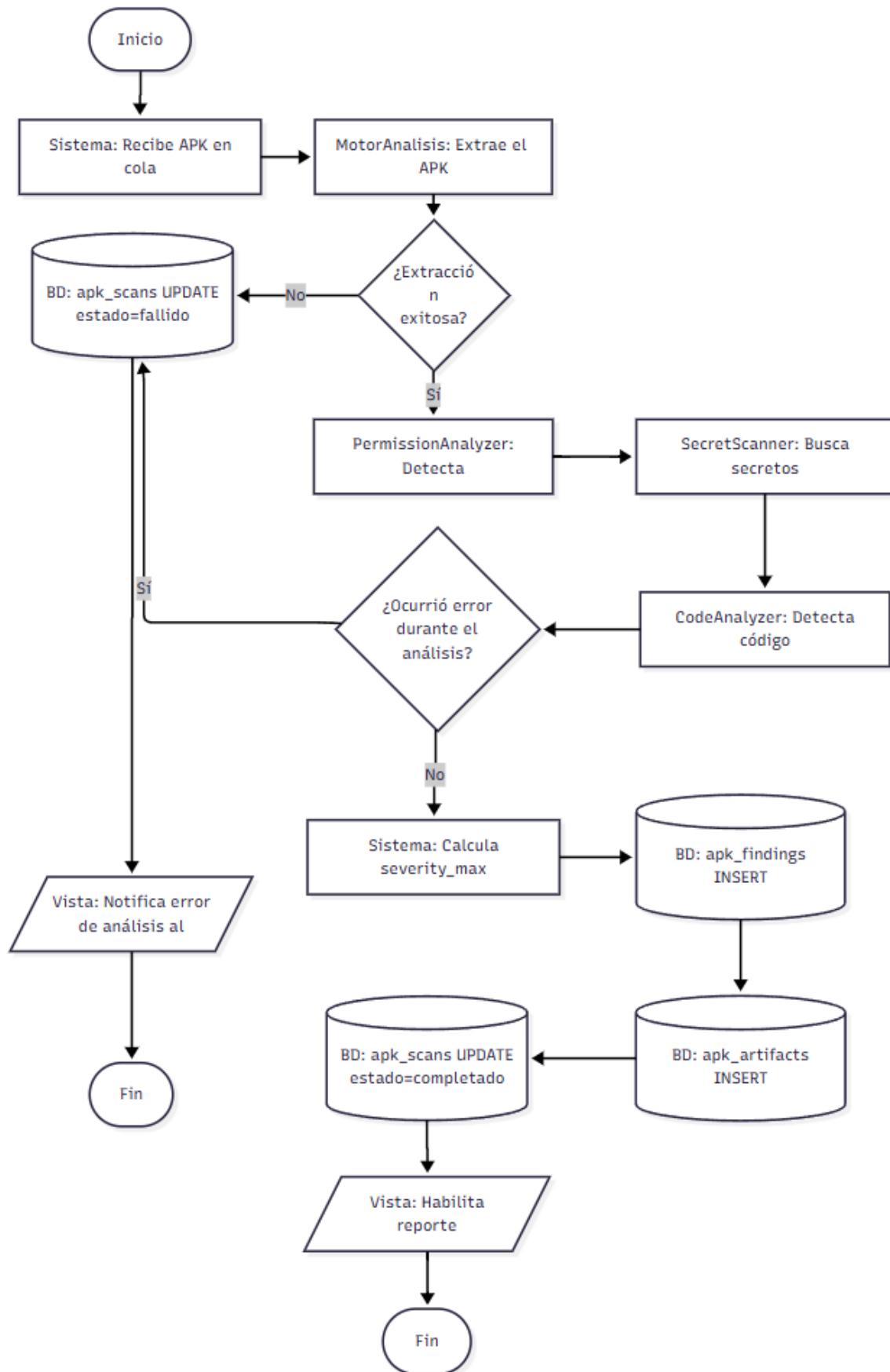
**CU-02: Iniciar Sesión**

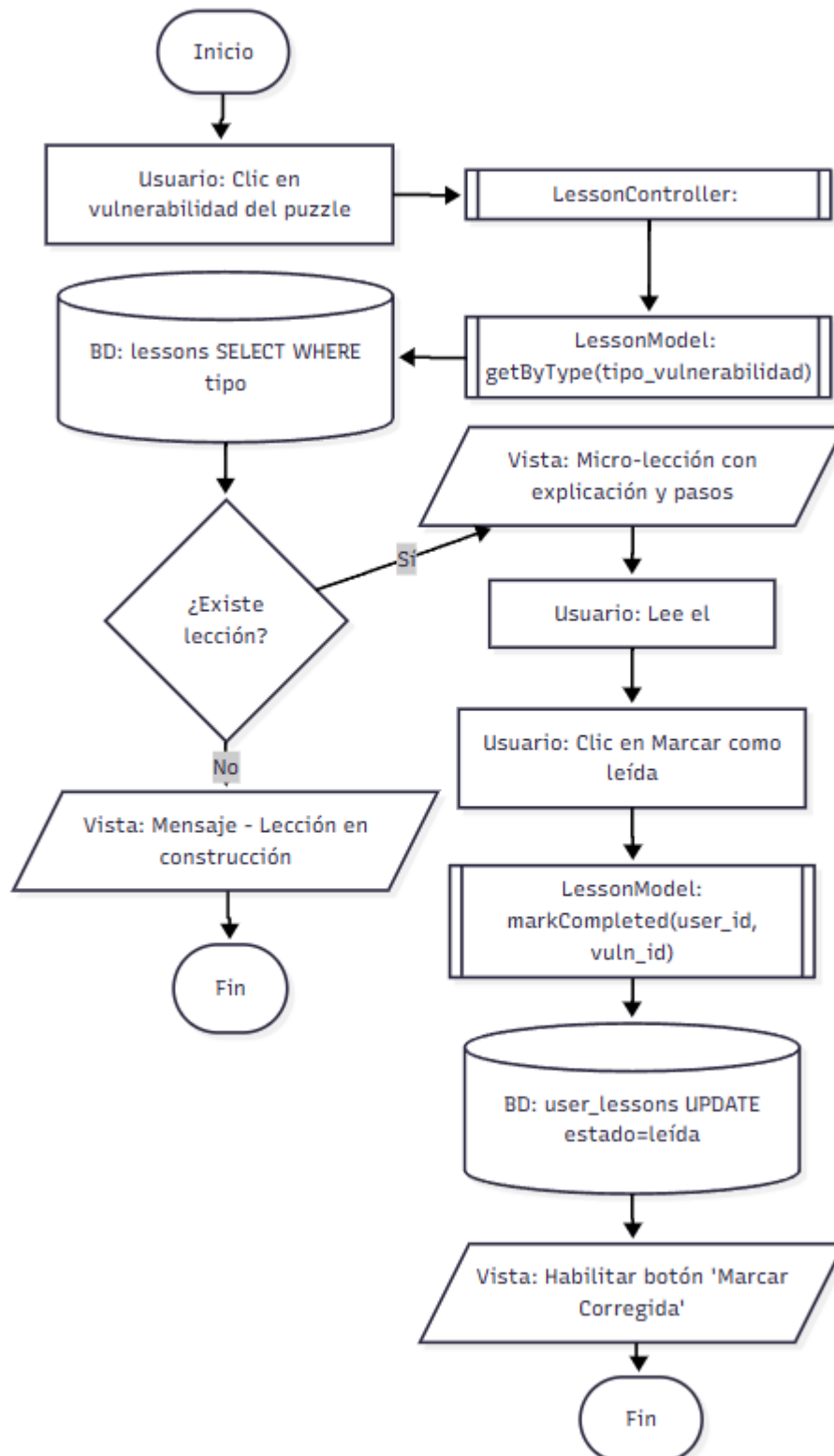


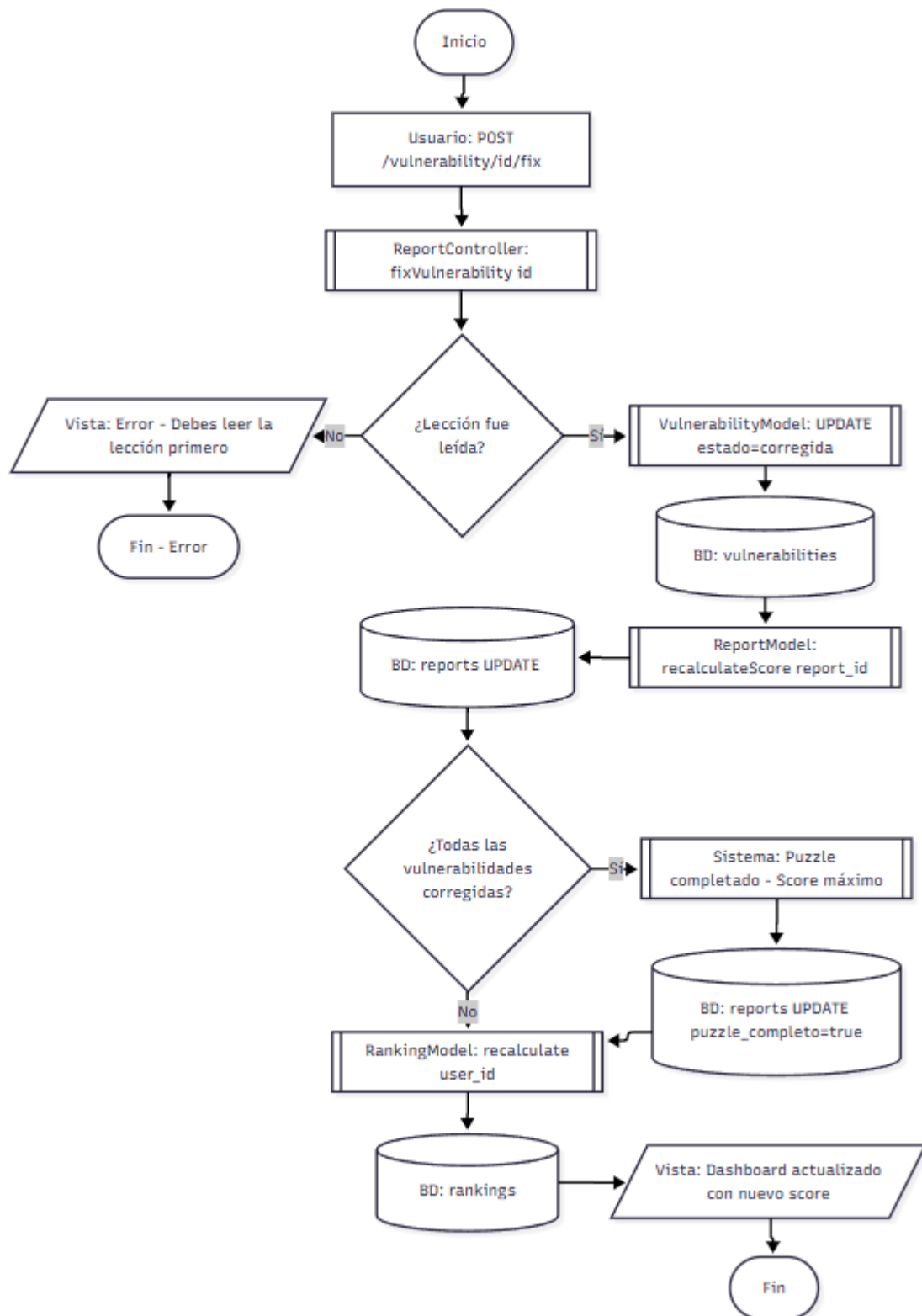
**CU-03: Subir APK para Análisis**

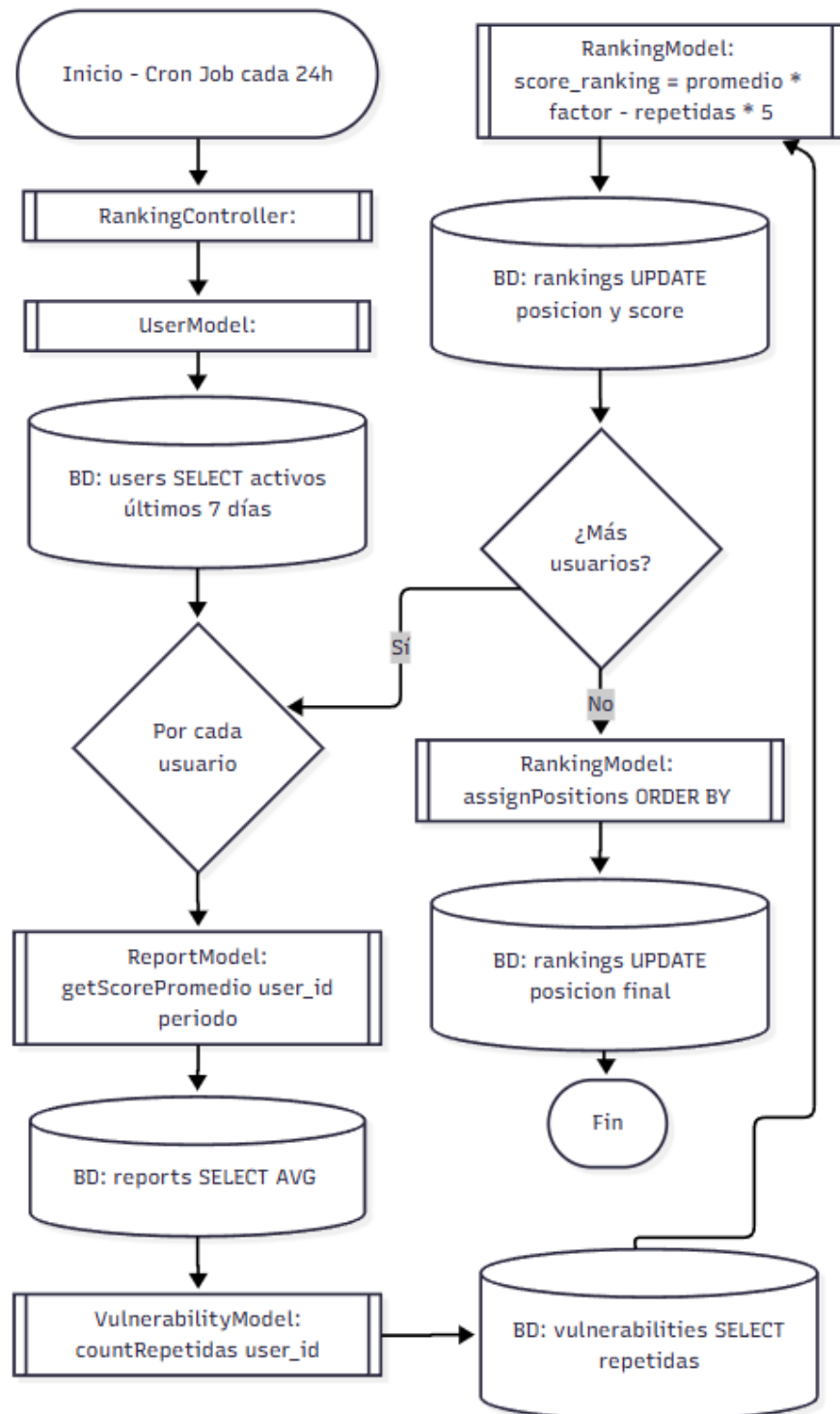


**CU-04: Ver Dashboard / Resultados del Análisis**

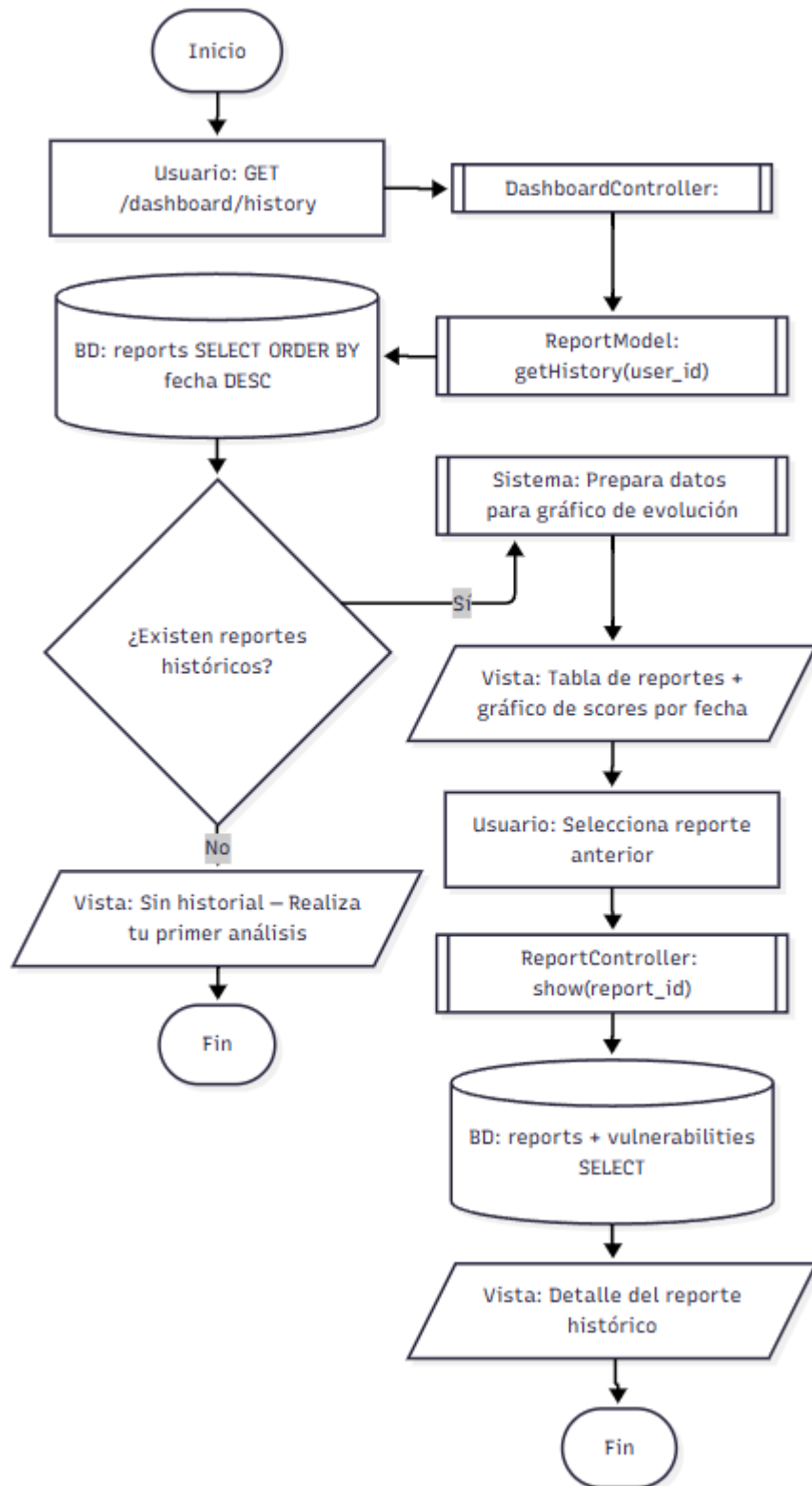
**CU-05: Ver Score de Seguridad**

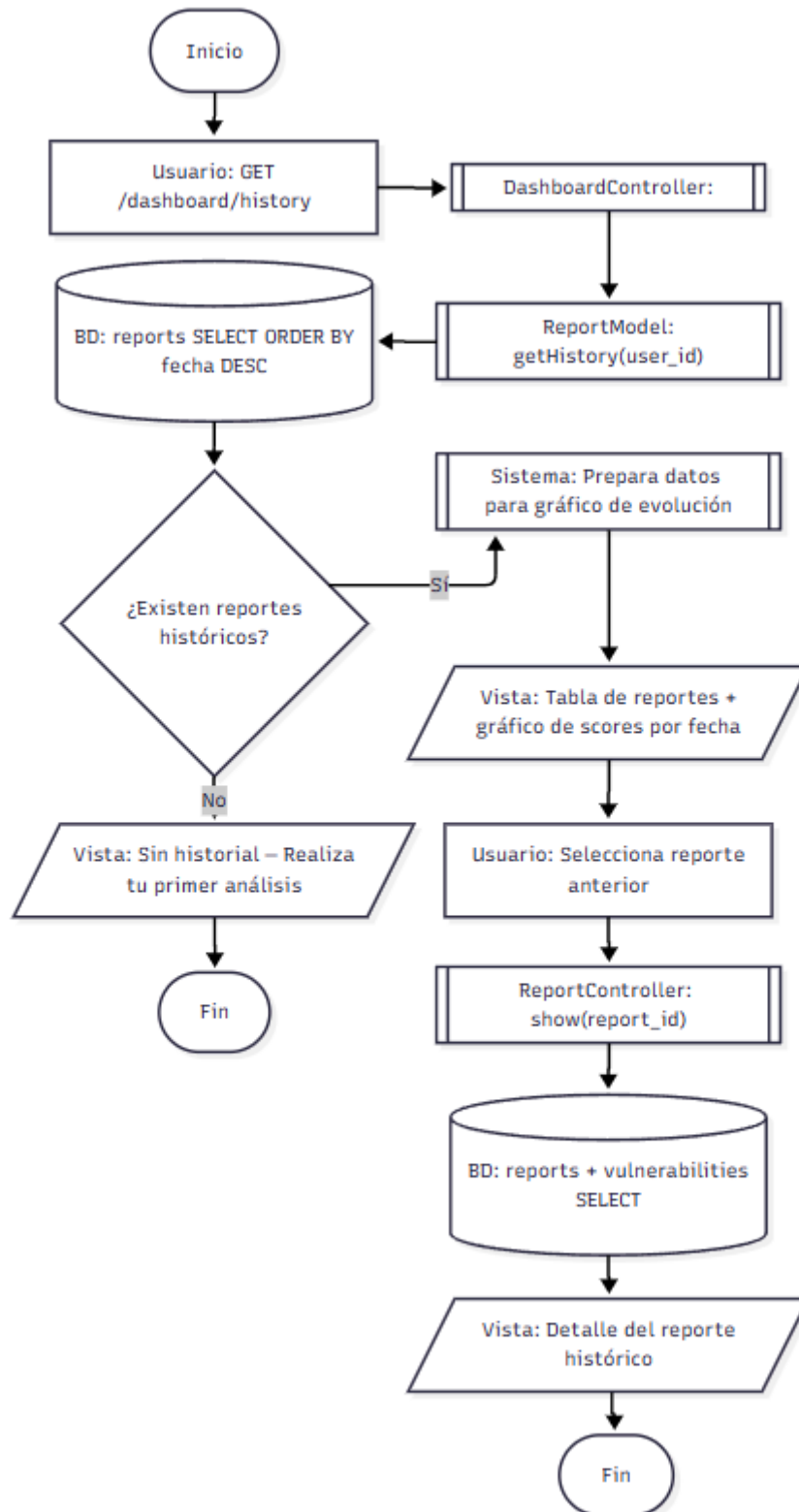
CU-06: Ver Detalle de Hallazgo

CU-07: Exportar Reporte PDF**CU-08: Motor IR — Desofuscación y Detección**



CU-09: Procesar y Persistir Hallazgos

**CU-10: Ver Historial de Escaneos**



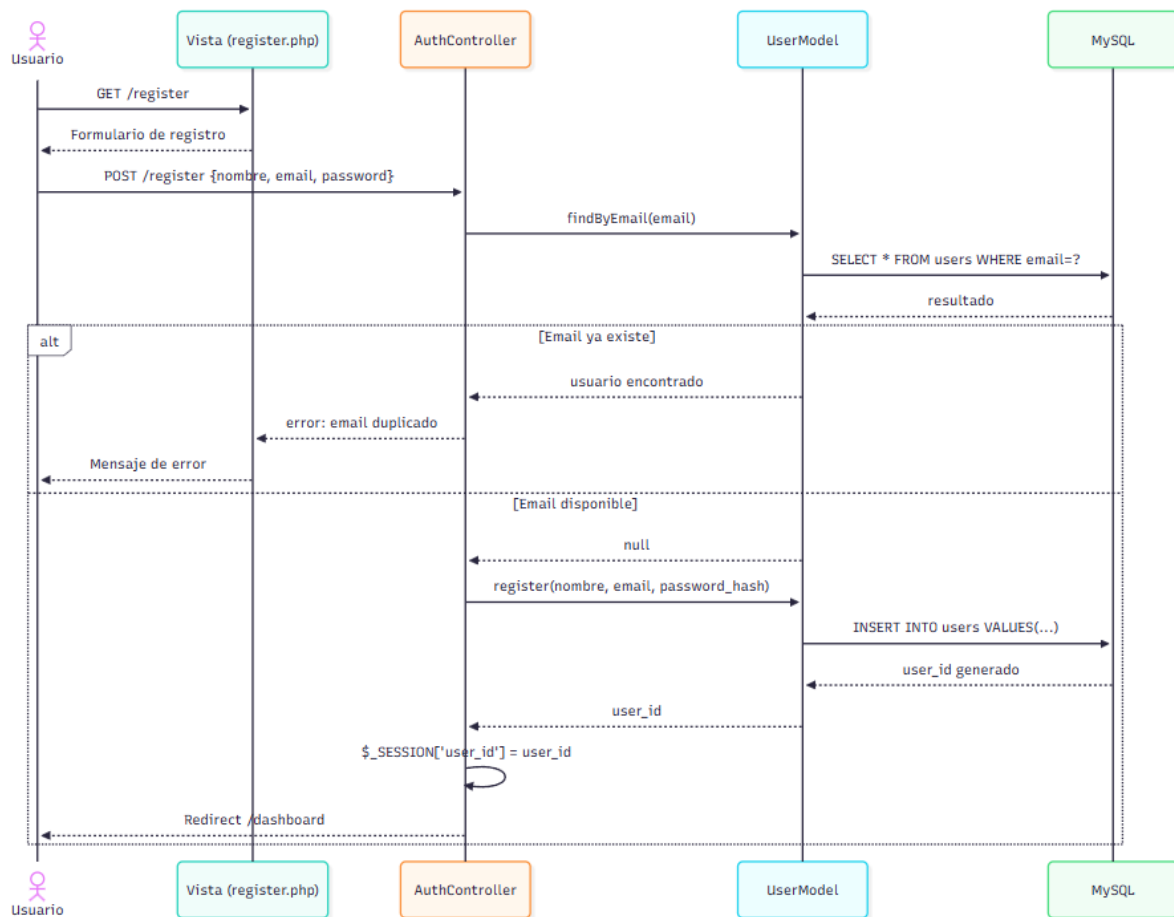
CU-11: Integración Empresarial API REST

CU-12: Agente Android — Escaneo Local del Dispositivo

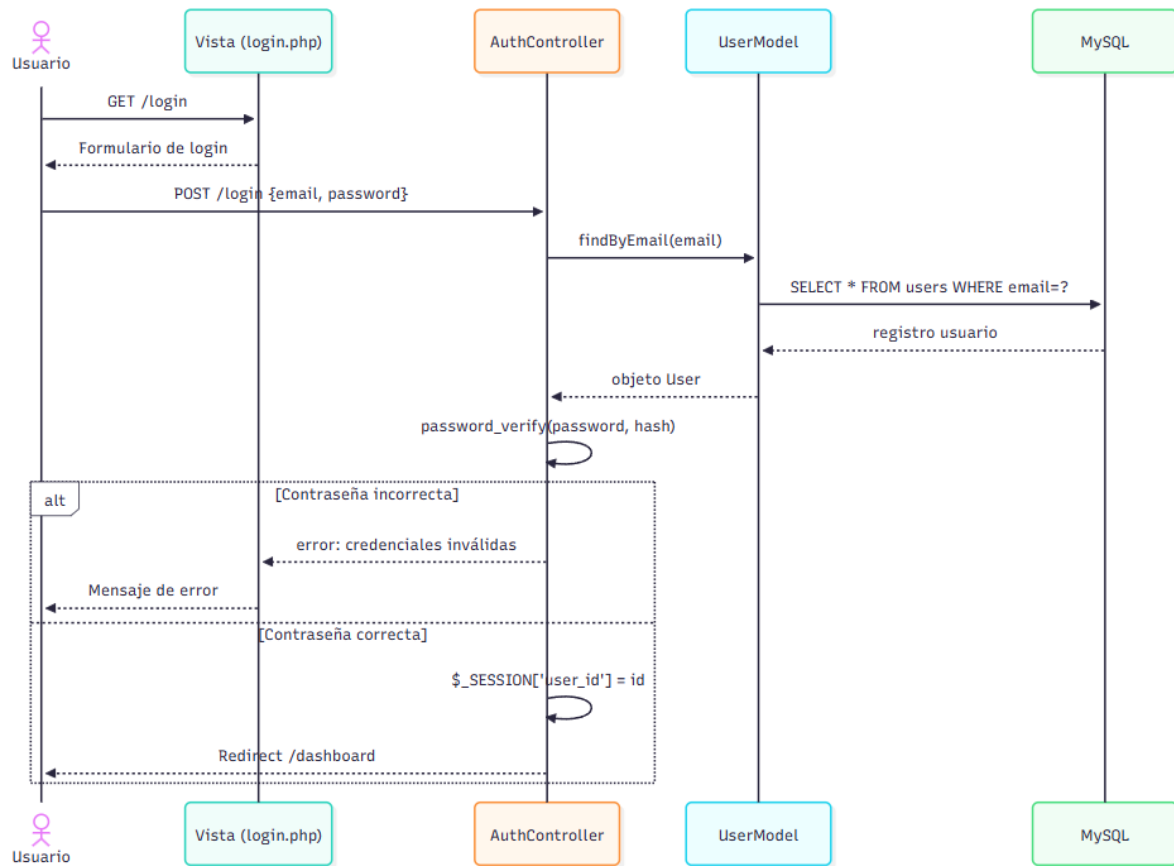
CU-13: Ver Comunidad en Tiempo Real

c. Diagrama de Secuencia

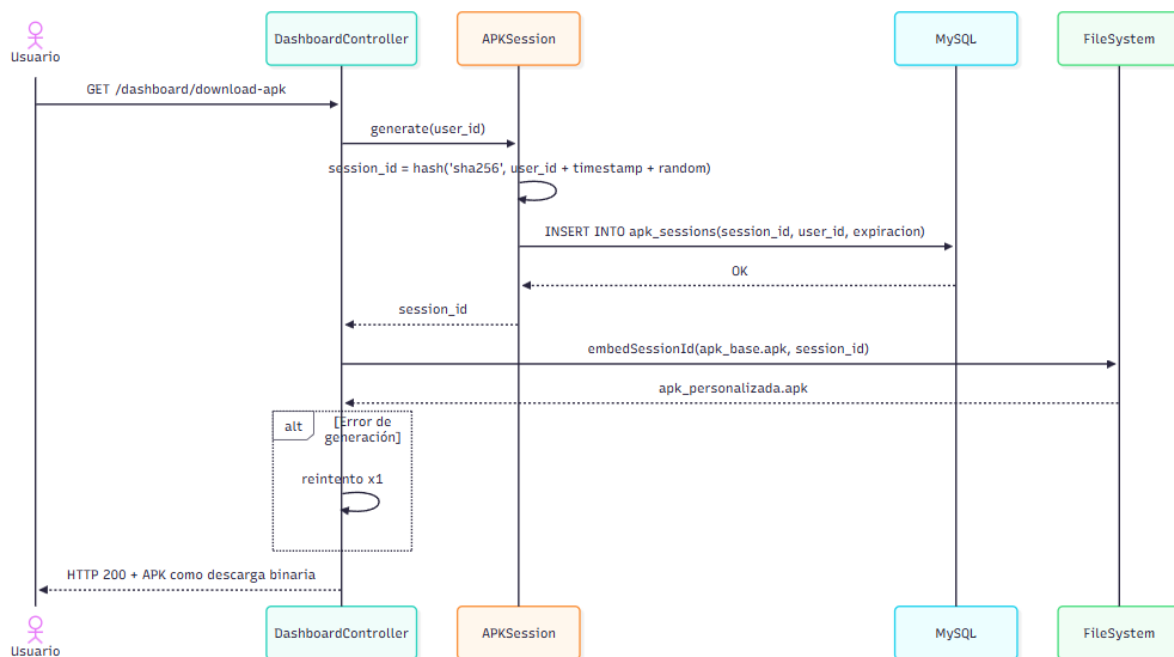
DS-CU01: Registrarse



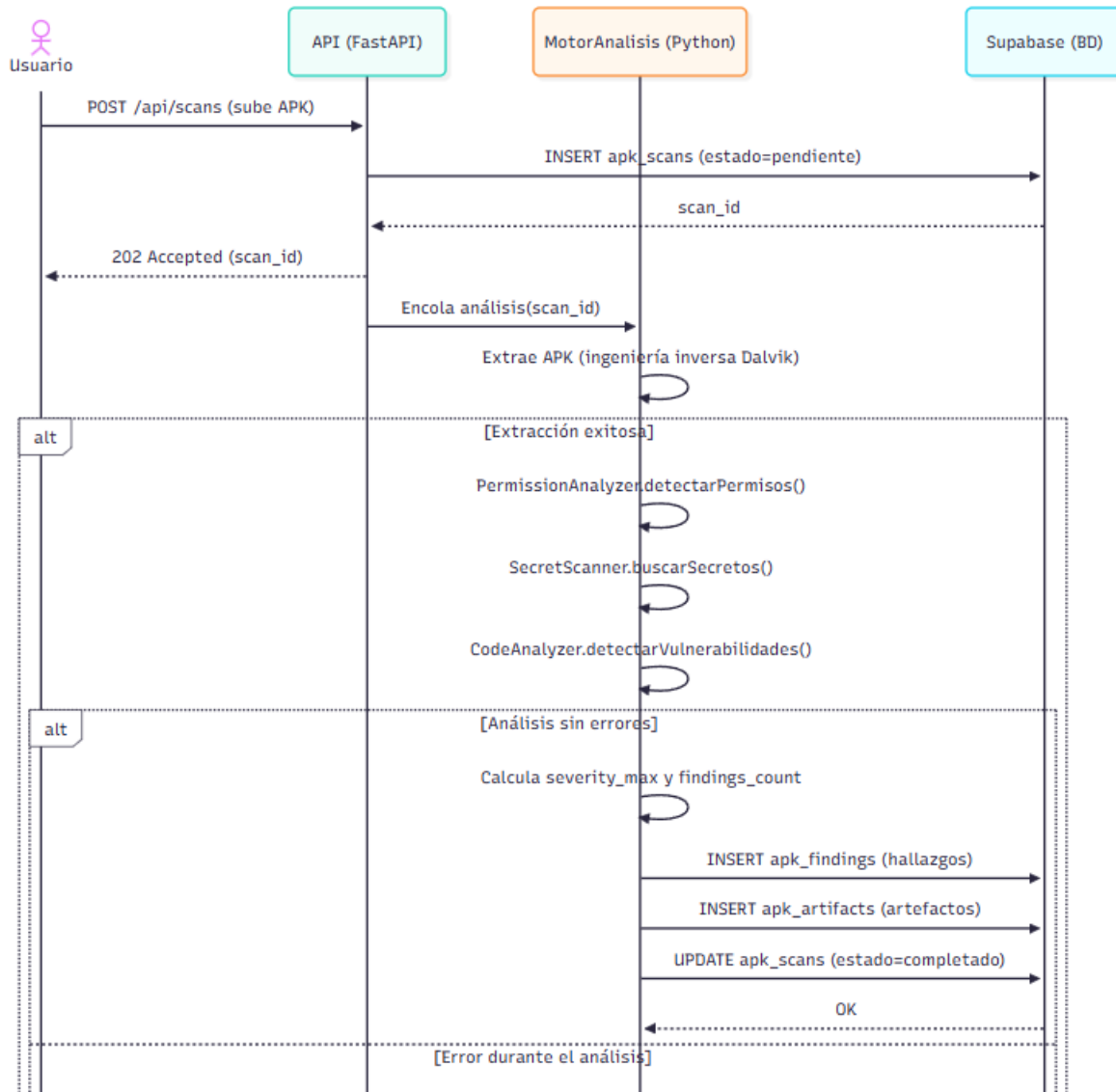
DS-CU02: Iniciar Sesión

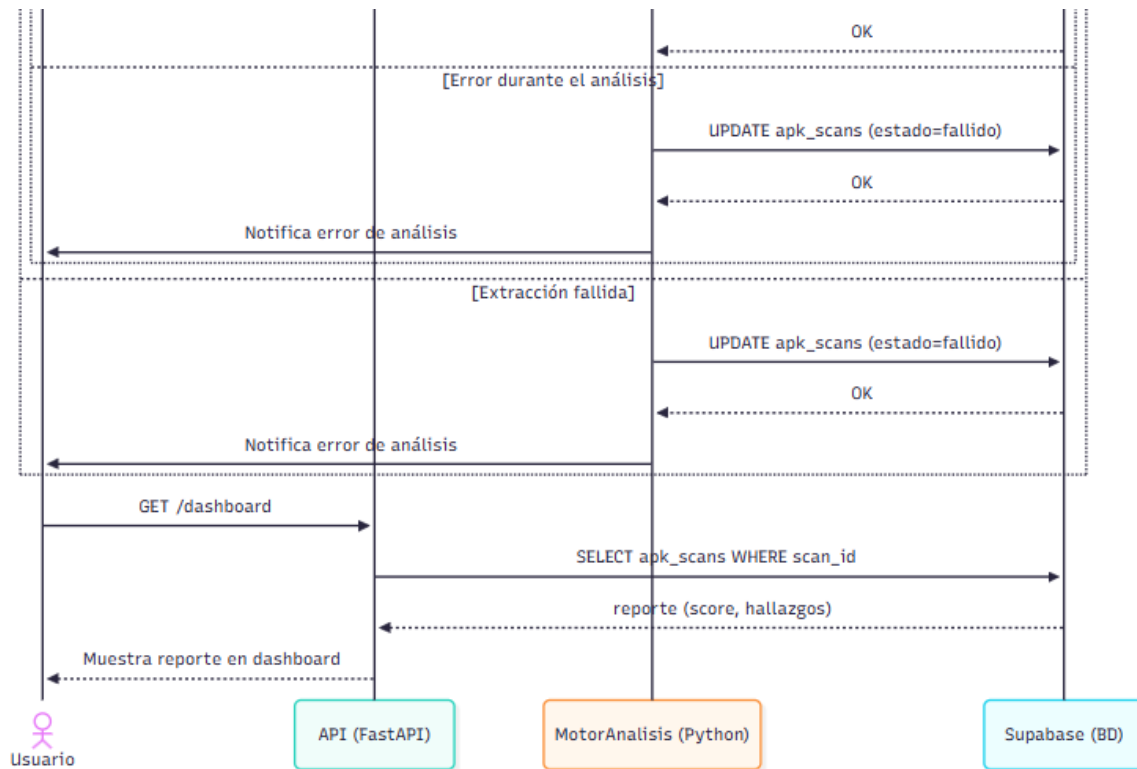


DS-CU03: Subir APK para Análisis

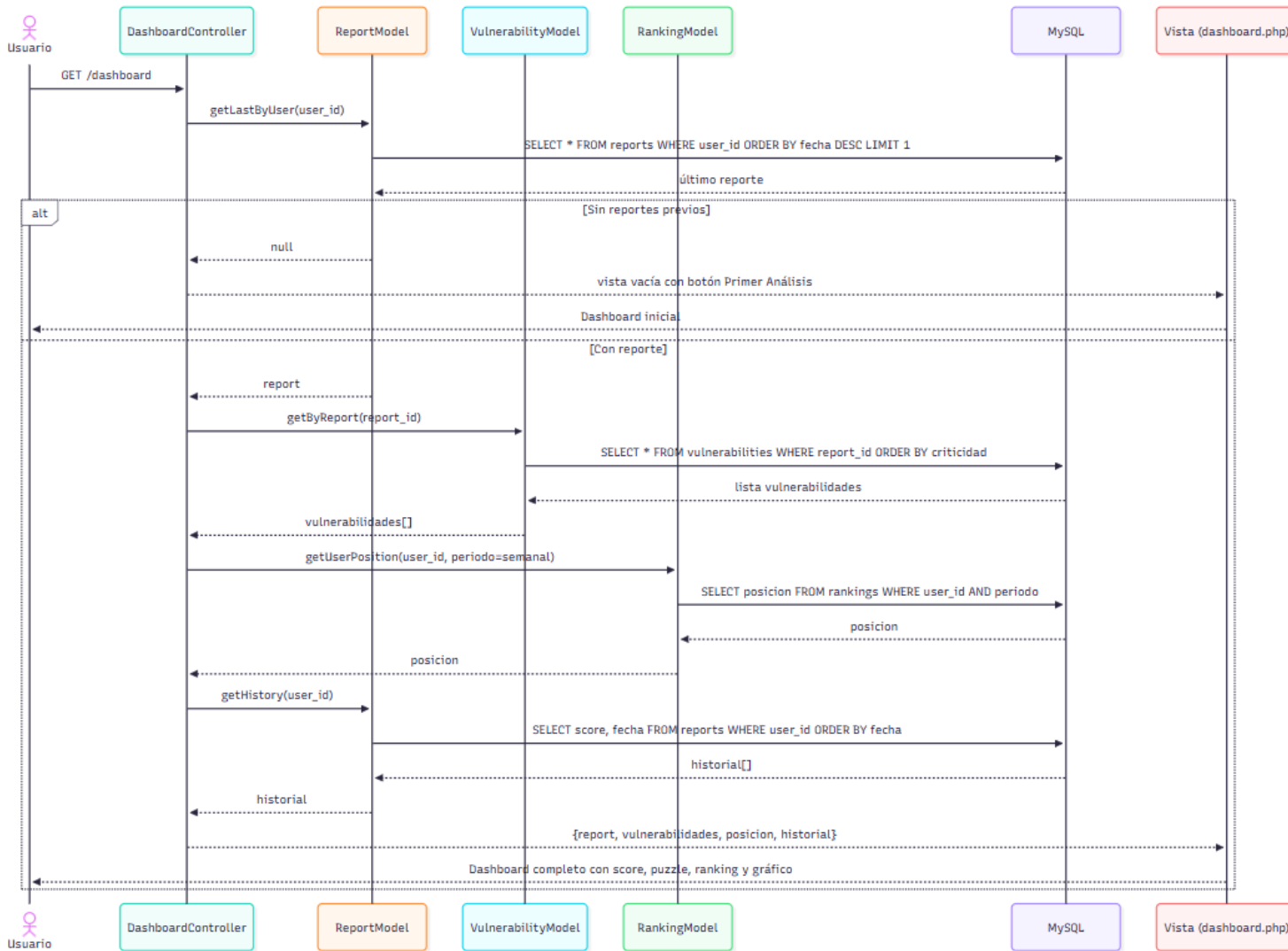


DS-CU04: Ver Dashboard / Resultados del Análisis

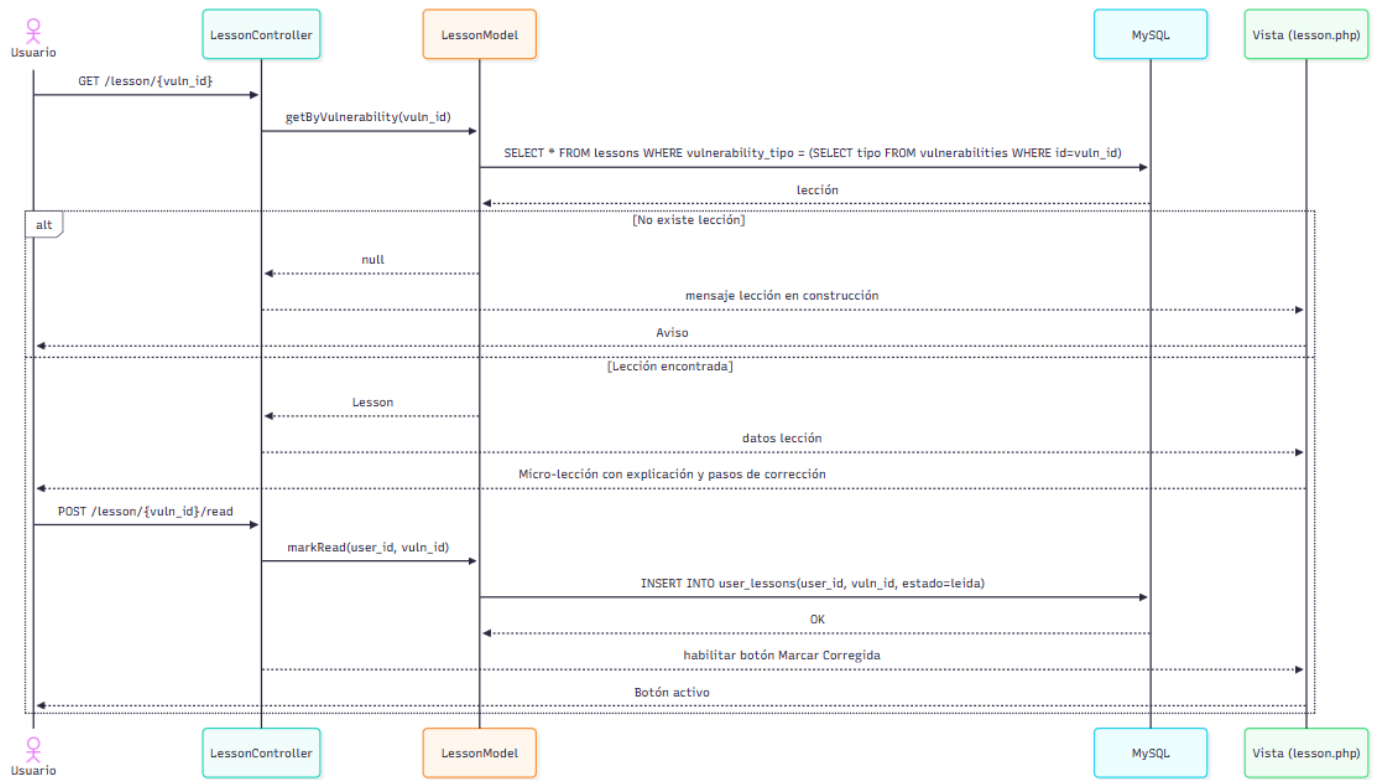




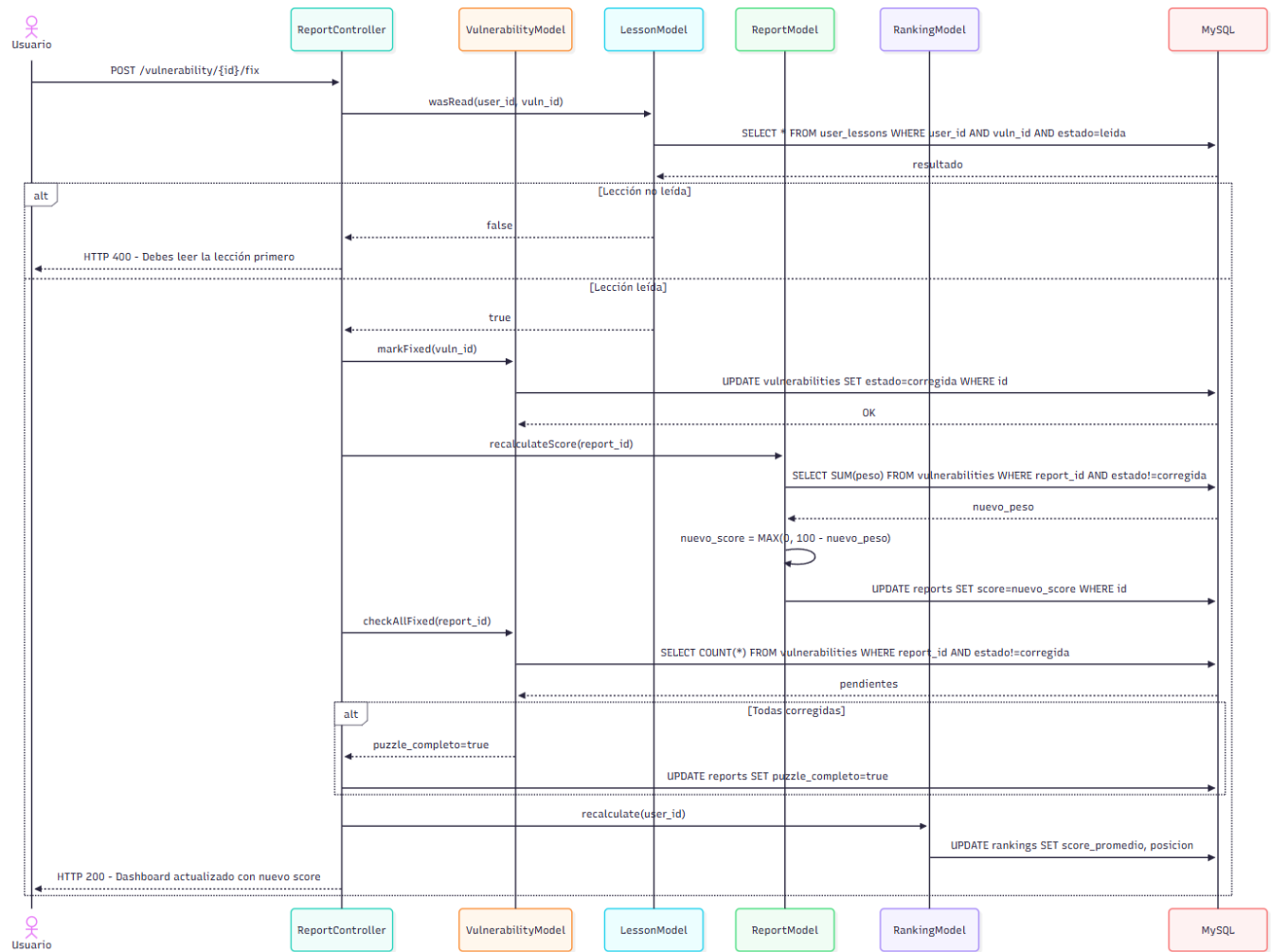
DS-CU06: Ver Detalle de Hallazgo



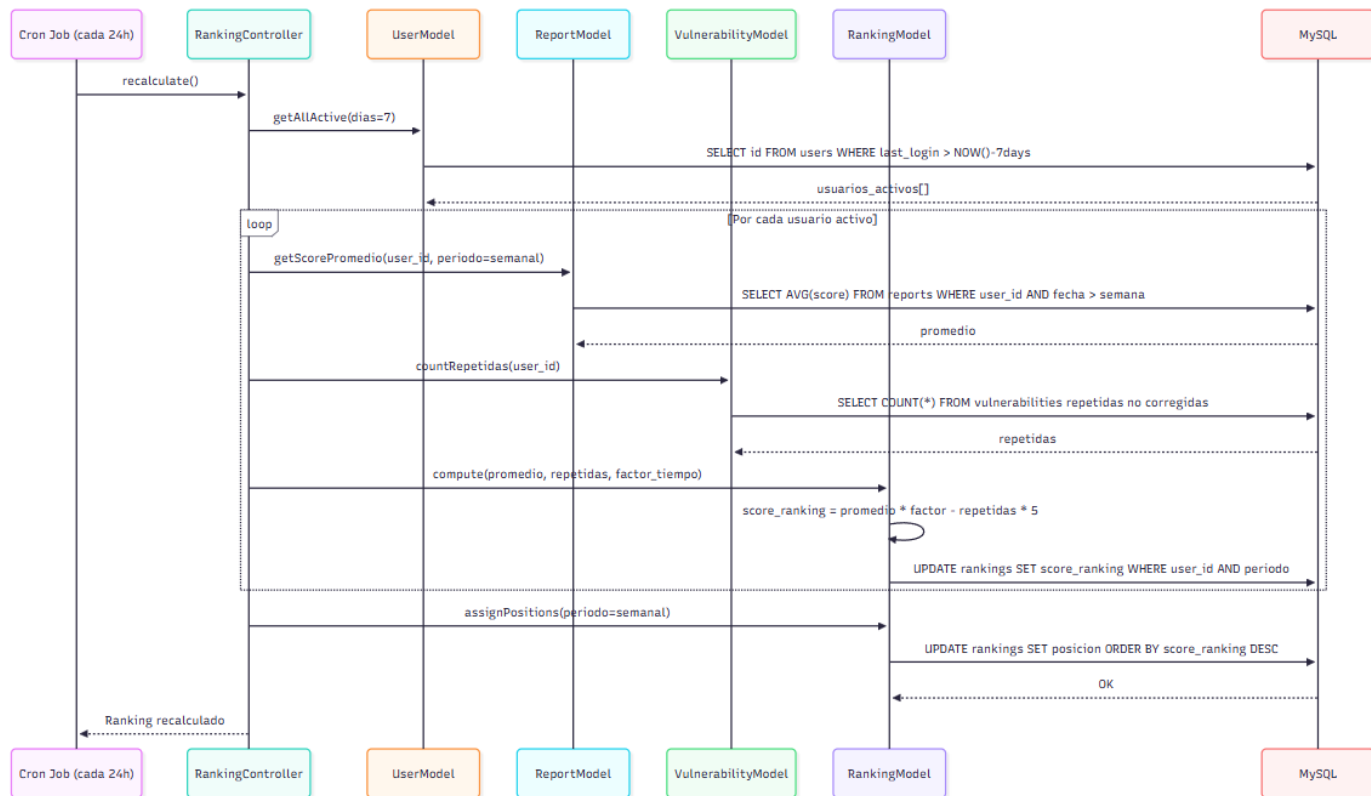
DS-CU07: Exportar Reporte PDF



DS-CU08: Motor IR — Desofuscación y Detección

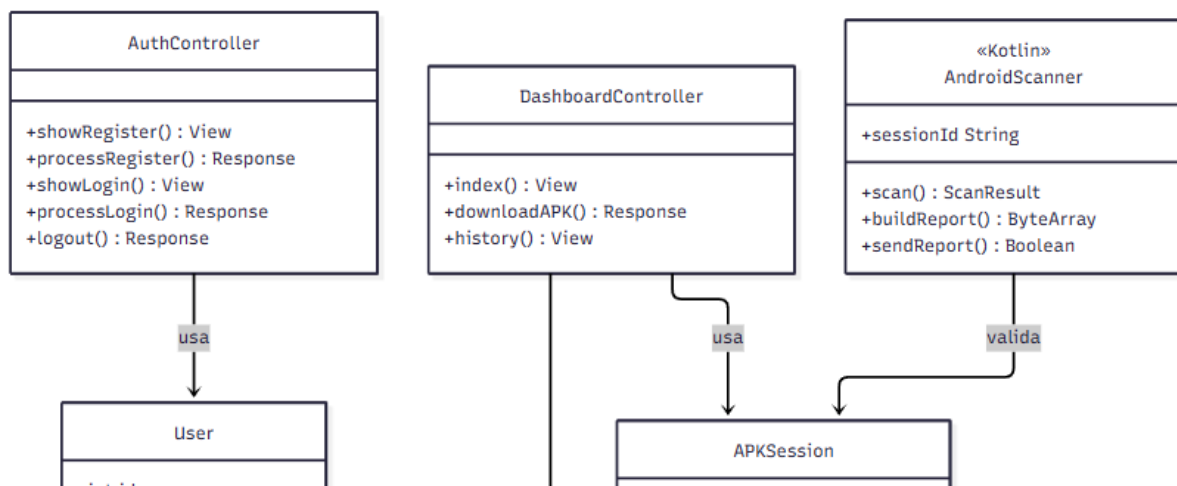


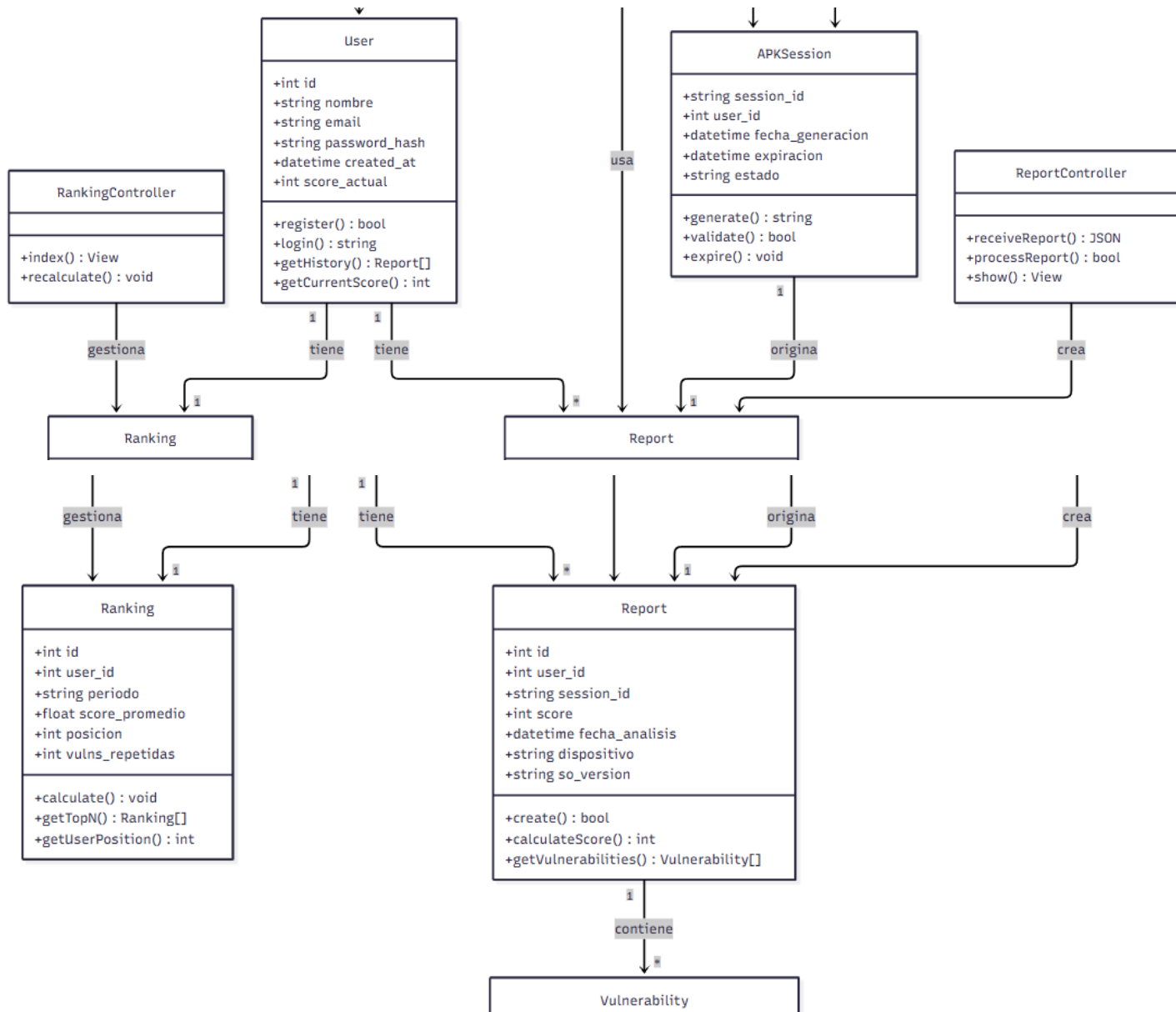
DS-CU09 a DS-CU13: Historial, API REST, Agente Android y Comunidad

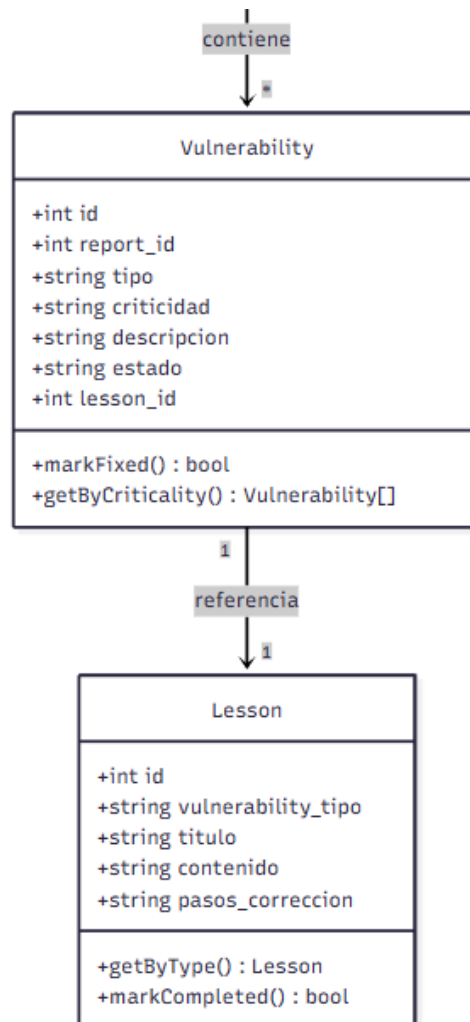


d. Diagrama de Clases

Diagrama de clases







CONCLUSIONES

- AnzenCore implementa un enfoque de análisis centralizado: el usuario sube su APK a la plataforma web y el servidor Python realiza el análisis completo, eliminando la necesidad de instalar software en el dispositivo del usuario.
- La arquitectura MVC en Python (FastAPI + Streamlit) garantiza separación de responsabilidades entre controladores, modelos y vistas, facilitando el mantenimiento y las pruebas de software.
- El motor de ingeniería inversa de AnzenCore aplica algoritmos de desofuscación sobre el bytecode Dalvik/DEX del APK, detectando vulnerabilidades incluso en aplicaciones ofuscadas con ProGuard/R8. Este enfoque nativo en Python stdlib (zipfile, struct, re) garantiza disponibilidad y control total del análisis sin depender de servicios externos de verificación de código o URLs.
- El agente móvil Android (Python/Kivy, API 26+) complementa el análisis del motor IR realizando escaneo local del dispositivo: permisos peligrosos de apps instaladas, certificados de red y configuración de seguridad del sistema. Envía el reporte

estructurado al backend vía API REST para persistencia y visualización en el dashboard del usuario.

- El uso de Supabase (PostgreSQL) como base de datos en la nube simplifica la persistencia y facilita el despliegue en Azure Container Apps sin necesidad de administrar infraestructura de base de datos.

RECOMENDACIONES

- Implementar pruebas unitarias y de integración desde el inicio del desarrollo (TDD) para garantizar la calidad del sistema, especialmente en los módulos de cálculo de score y procesamiento de reportes.
- Realizar pruebas de penetración (pentesting) sobre la API REST para verificar la seguridad del endpoint de recepción de reportes antes del despliegue.
- Documentar los casos de prueba según el estándar IEEE 829, alineados con los requerimientos funcionales definidos en este documento.
- Considerar en iteraciones futuras la integración con bases de datos de vulnerabilidades CVE para enriquecer el análisis de apps instaladas.
- Evaluar la publicación en F-Droid (repositorio de apps Android de código abierto) para aumentar el alcance del proyecto.

BIBLIOGRAFÍA

- Google LLC. (2024). Android Developer Documentation — Security best practices. <https://developer.android.com/privacy-and-security/security-tips>
- C4 Model (2024). The C4 model for visualizing software architecture. Recuperado de: <https://c4model.com/>